Part-of-speech Tagging & Parsing

Sean MacAvaney & Jake Lever

University of Glasgow | School of Computing Science

Welcome!

Today's lecture will introduce:

- Background on Natural Language Processing
- Sequential labelling: Parts-of-Speech, Named Entity Recognition
  - Classification techniques: HMM, Viterbi Algorithm
- Tree Structures: Constituency and Dependency Syntax Trees
  - Parsing techniques: Transition-Based Parsing

Let's say we want to understand this text:

John drank some wine on the new sofa. It was red.

Previously we covered:

- Bag-of-words representations:

  `{john, drank, some, wine, on, the, new, sofa, it, was, red}`

  (also with weighting schemes, like TF-IDF)

- N-gram language models:

  `P("John drank some wine on the new sofa. It was red.")`

  `P("red" | "John drank some wine on the new sofa. It was")`
  `≈`
  `P("red" | "It was")`

  (with Markov Assumption)

These approaches are rather "stupid", right?
They do not actually "understand" what's happening.

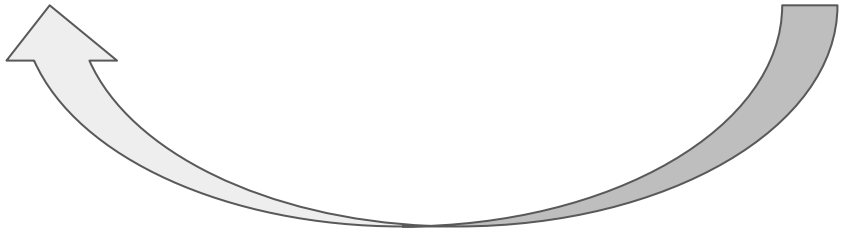
Let’s say we want to understand this text:

John drank some wine on the new sofa. It was red.

A self-contained phrase
(Noun Phrase)

Let’s say we want to understand this text:

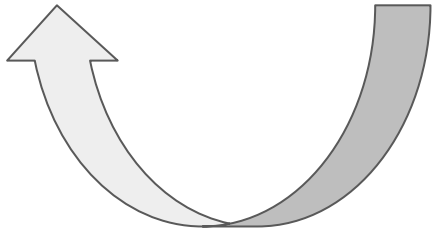
John drank some wine on the new sofa. It was red.



Reference (anaphora)

Let’s say we want to understand this text:

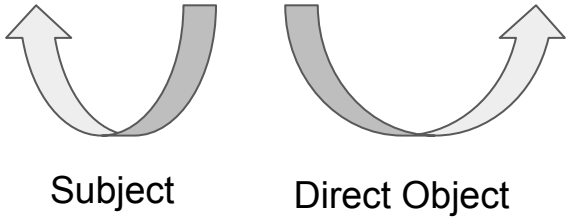
John drank some wine on the new sofa. It was red.



Or…? Reference is ambiguous.

Let’s say we want to understand this text:

John drank some wine on the new sofa. It was red.



Let’s say we want to understand this text:

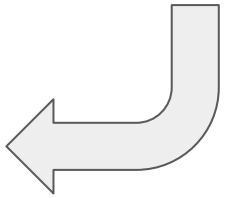
John drank some wine on the new sofa. It was red.

Modifies where the drinking was done. (Prepositional phrase.)

Let’s say we want to understand this text:

John drank some wine on the new sofa. It was red.

Implication: John spilled the wine on the new sofa.



Let's say we want to understand this text:

John drank some wine on the new sofa. It was red.

There are lots of relationships between words/phrases in text beyond sequential words.

To really *understand* text, it's important to be able to model these relationships.

Background of Natural Language Processing

Text is more than a bag-of-words

What does NLP involve?

“Rachel ran to the bank. It was closed”

created with Copilot

- Syntax - how words combine to form grammatical structure
  - e.g. bank is a direct object
- Semantics - how words convey meaning
  - e.g. bank refers to the financial institution
- Pragmatics - how context contributes to meaning
  - e.g. using context to understand “it”

Context Dependence and Background Knowledge

Rachel **ran** to the **bank**.

vs.

Rachel **swam** to the **bank**.

John drank some **wine** at the **table**. **It** was **red**. vs.

John drank some wine at the **table**. **It** was **wobbly**.

Real newspaper headlines

“Boy paralyzed after tumor fights back to gain black belt”

“Scientists study whales from space”

“Juvenile court to try shooting defendant”

Garden Path Sentences

Sentences that do not appear to be syntactically correct on the first pass.

- The old man the boat.

- The complex houses married and single soldiers and their families.

- The horse raced past the barn fell.

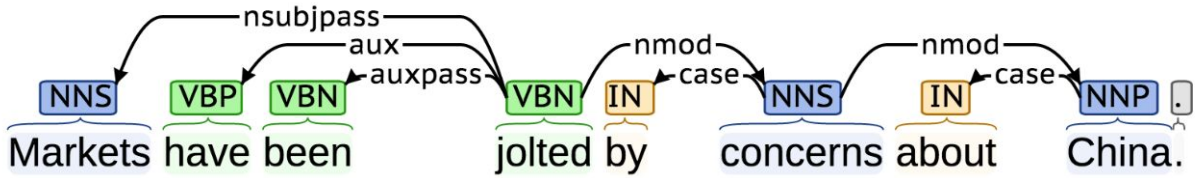
A typical NLP pipeline

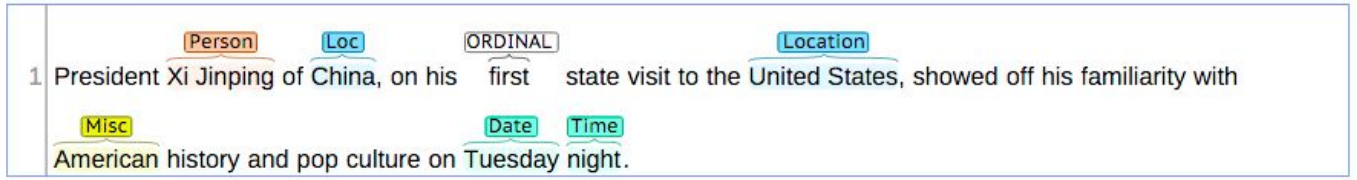
- Tokenization and lemmatization
  - Not as straightforward as you might think
- Sentence boundary detection
  - Most NLP operates on individual sentences.
- **Part-of-speech-tagging**
  - Identify nouns, verbs, adjectives, etc…
- **Parsing (dependency)**
  - Diagramming sentences
- Named Entity Recognition
  - Detect and classify entities
- Coreference resolution
  - Resolve pronouns to named entities

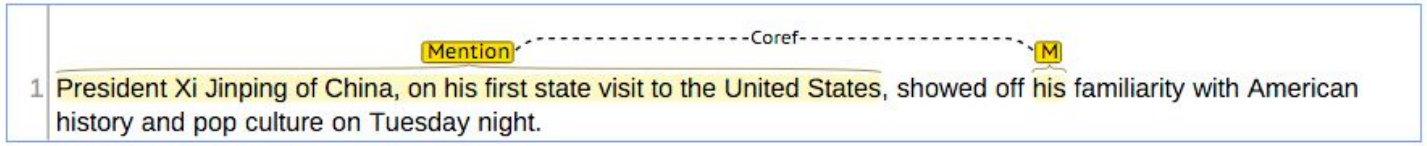
"Mr. Mackintosh went shopping. He…"					
Mr.	Mackintosh	went	shopping	.	He

Mr.	Mackintosh	went	shopping	.	He
SS	I	I	I	ES	SS

A	tall	boy	ran	home	.
DT	JJ	NN	VBD	NN	.



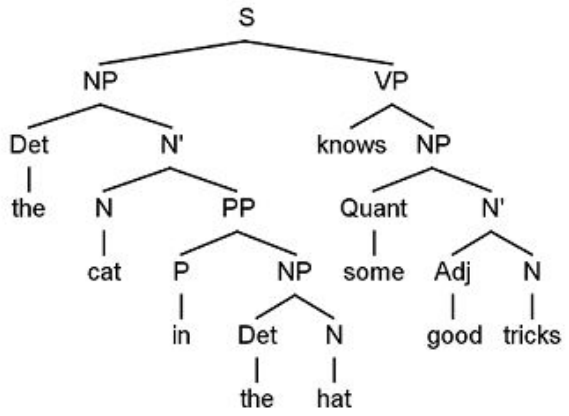




How is all this done?

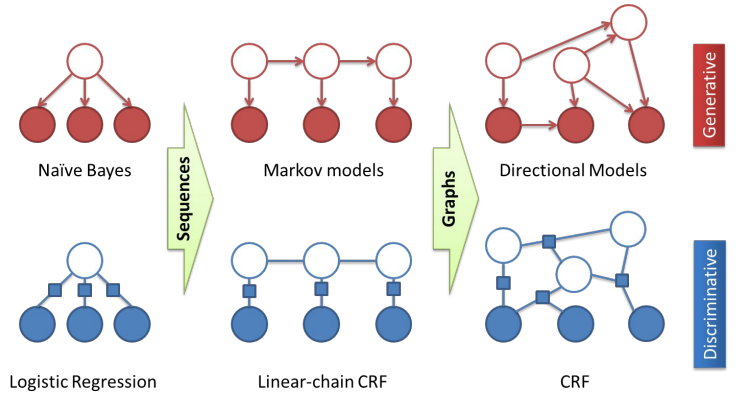
Rule-based approaches

- Linguistic + Domain knowledge
- Example: tokenization - re.split(r‘\w+’, raw)
- Maintaining/updating rules manually is difficult.



Statistical methods for identifying patterns

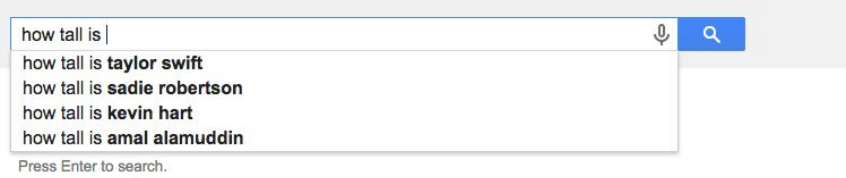
- Supervised machine learning



Adapted from C. Sutton, A. McCallum, “An Introduction to Conditional Random Fields”, ArXiv, November 2010

Learn from human interactions with computers

- Mainly as a way to acquire training data



Sequential NLP Structures

Sequential NLP Structures

Some NLP information can be formulated as a sequence of labels:

- Part-of-Speech Tagging
- Named Entity Recognition
- Word/Structure Segmentation

In this setting, each unit (token, character, sentence) is assigned a label.

The labels are not independent; they depend on the surrounding labels.

Parts-of-Speech (POS)

A very basic syntactic analysis.

Every word is assigned a tag based on its grammatical role.

E.g., Noun, Verb, Adjective, etc.

John	drank	some	wine	on	the	new	sofa	.	It	was	red	.
PROPN	VERB	DET	NOUN	ADP	DET	ADJ	NOUN	PUNCT	PRN	AUX	ADJ	PUNCT

Parts-of-Speech (POS)

Many different granularities – some specific to certain languages.
Generally a reasonable place to start: “Universal” POS tags:

Tag	Name	Example(s) - Context-dependent
ADJ	adjective	big, old, green
ADP	adposition (preposition)	in, to, during
ADV	adverb	very, well, exactly
AUX	auxiliary verb	is, has, must
CCONJ	coordinating conjunction	and, or, but
DET	determiner	a, the, some
INTJ	interjection	psst, ouch, hello
NOUN	noun	girl, cat, tree

Tag	Name	Example(s) - Context-dependent
NUM	numeral	five, 70, IV
PART	particle	‘s, not
PRON	pronoun	he, her, myself, everybody
PROPN	proper noun	Mary, Glasgow, HBO
PUNCT	punctuation	. , ( )
SCONJ	subordinating conjunction	that, if, while
SYM	symbol	$ 😝 ♥‿♥ *1-800-COMPANY*
VERB	verb	drank, running, sits

http://universaldependencies.org/u/pos/

Parts-of-Speech (POS)

- State-of-the-art techniques achieve > 98% accuracy on news
  - Not a PhD topic anymore!
- Average POS tagging disagreement amongst expert human judges for the Penn Treebank was 3.5%
- POS tagging in ‘low resource’ languages is harder (~87%)
  - Languages without a lot of training data
- POS tagging for Tweets is harder (state-of-the art ~ 88 %)

Other Sequence Tagging Problems

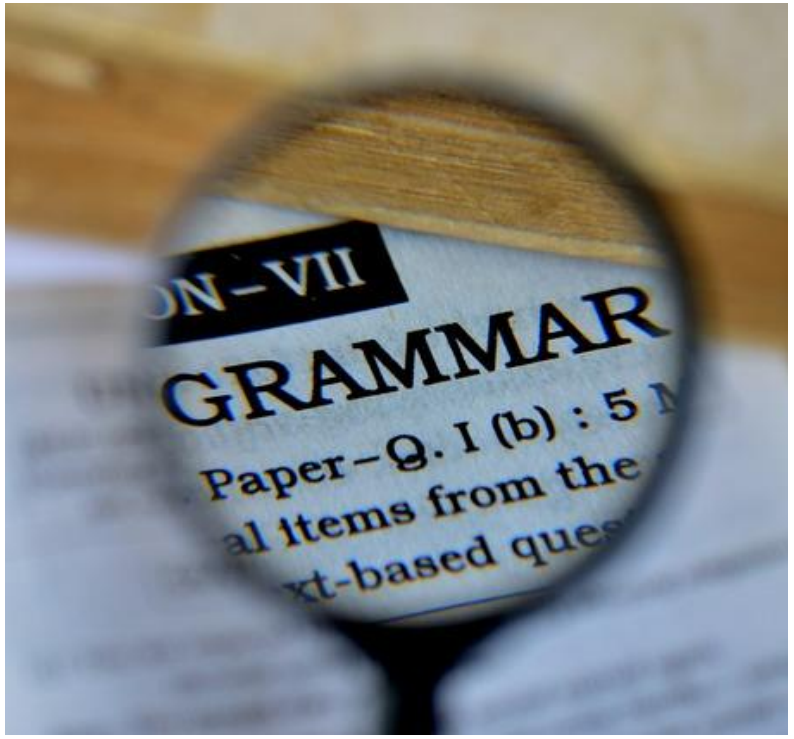
Word Segmentation

B	B	I	I	B	I	B	I	B	B
而	相	对	于	这	些	品	牌	的	价

Structure Segmentation

Sequential Labelling

How to approach sequence labelling



Build a system that can build these structures automatically.

- Rule-based: Error-prone; building & maintaining rules is challenging!
- Supervised Learning: Most common/effective approach.

Can learn from *training sequences* from labelled *treebanks*.

We'll cover Hidden Markov Models – more recent approaches exist based on neural networks, but many of the same ideas apply.

Naïve Bayes

From Bayes Rule:

$$P(C_k|w) \propto P(C_k)P(w|C_k)$$

Prob. of POS tag given word

Prob. of POS tag

Prob. of word given POS tag

Problem: Ignores sequential information! Words will always be assigned the same POS tag, regardless of how they were used in the sentence.

Example:

$$P(\text{VERB}|\text{"call"}) \propto P(\text{VERB})P(\text{"call"}|\text{VERB})$$

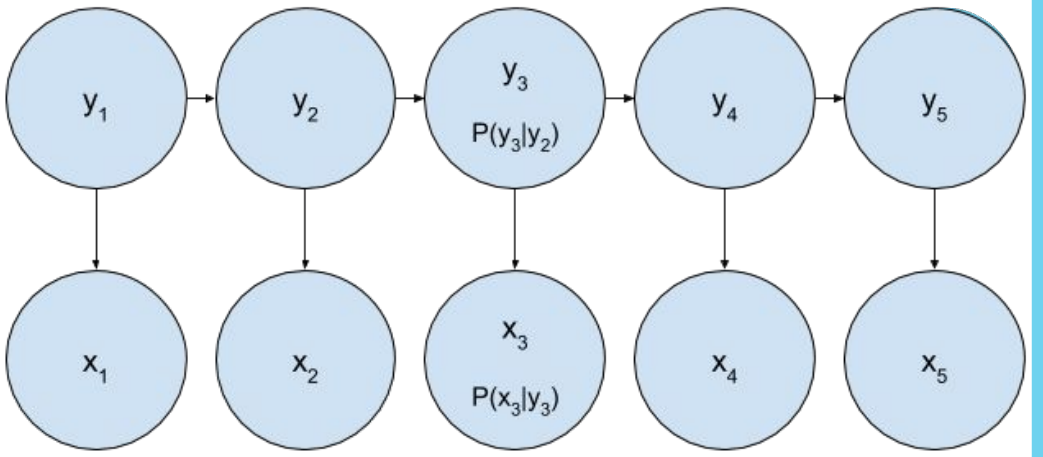
"That was a bad **call**"

"Please **call** your mum after school."

$$P(\text{VERB}|\text{"call"}) \propto P(\text{VERB})P(\text{"call"}|\text{VERB})$$

Hidden Markov Model

- Generalization of Naïve Bayes to sequences
- Assume an underlying set of ***hidden*** (unobserved) states (labels), ***y***, in which the model can be.
- Assume probabilistic **transitions between states** over time as sequence is generated.
- Assume a ***probabilistic*** generation of tokens from states (e.g. words generated for each POS).
- Assume current state is dependent only on the previous state. (known as the '**Markov** assumption')
  - The assumption may vary (2nd order = previous 2 states)



Parts of the model:

$\mathbf{x_i}$: observed nodes (i.e. words)

$\mathbf{y_i}$: hidden nodes (i.e. POS tags)

- goal will be to predict these

Transitions: $\mathbf{p(y_i \mid y_{i-1})}$ *(horizontal arrows)*

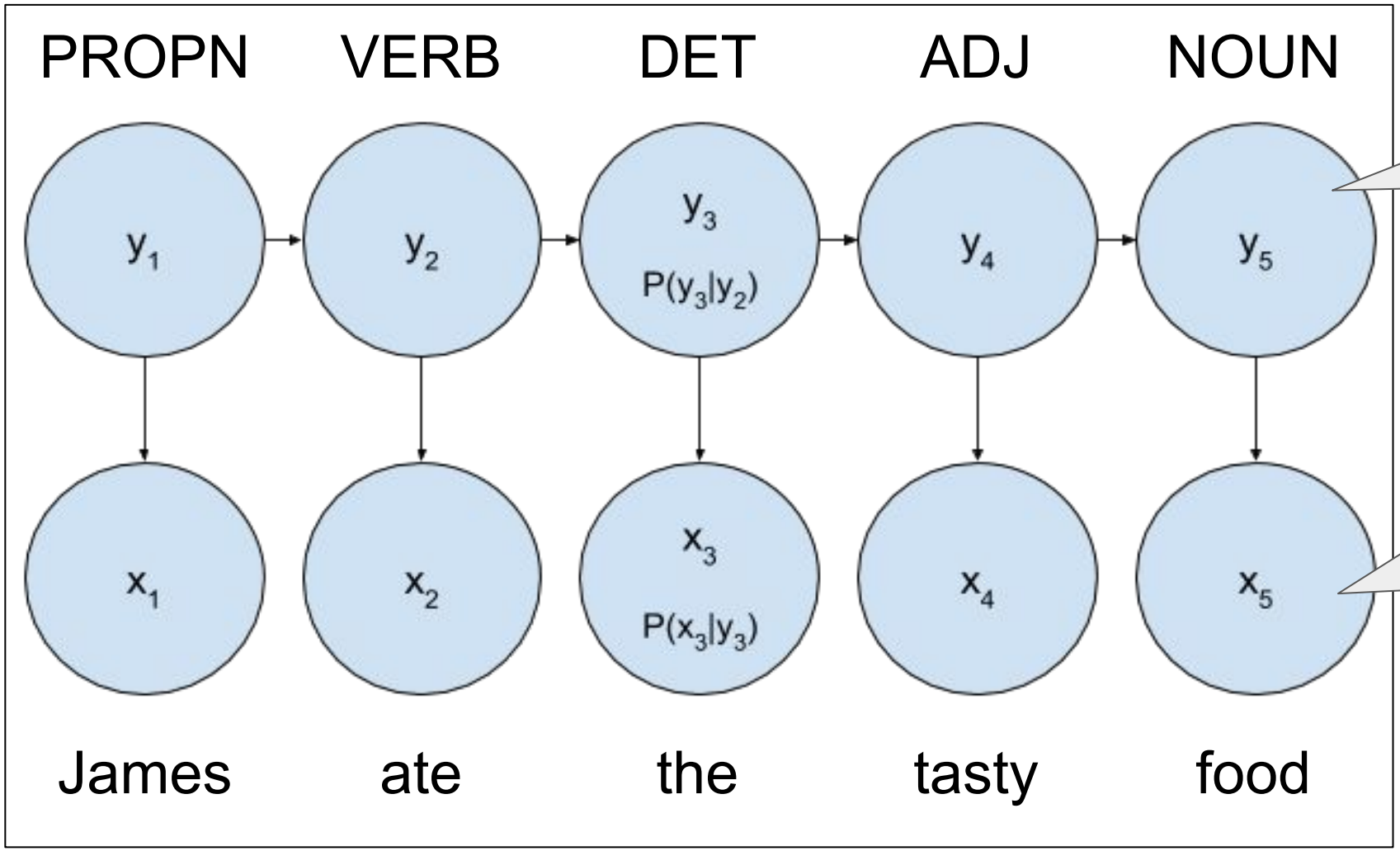
Emissions: $\mathbf{p(x_i \mid y_i)}$ *(vertical arrows)*

Things to Do:

Training: learn $p(y_i \mid y_{i-1})$ and $p(x_i \mid y_i)$

Inference (test) : given x_i , infer y_i or $p(y_i)$

POS tagging with HMMs

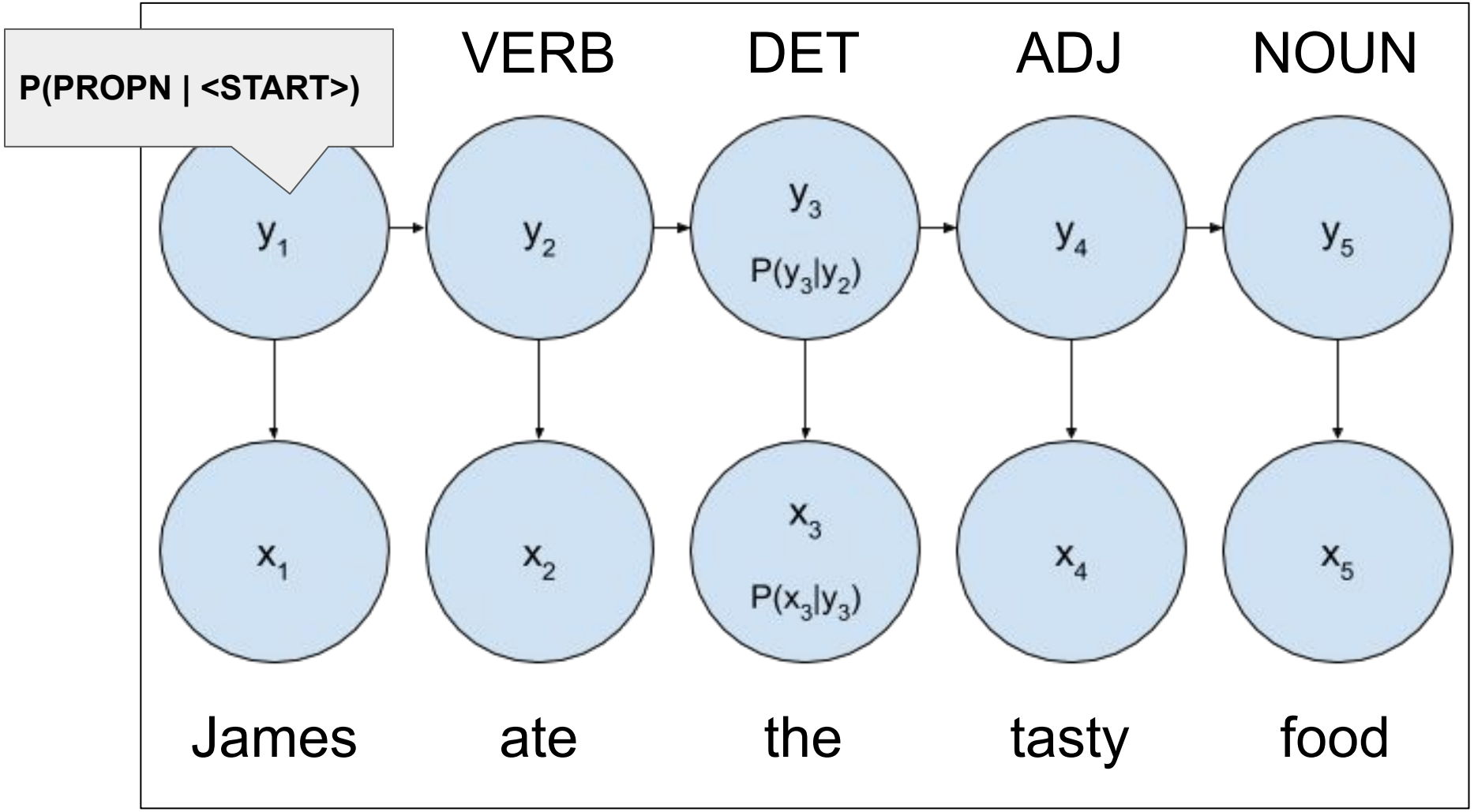


Hidden States: The POS tags

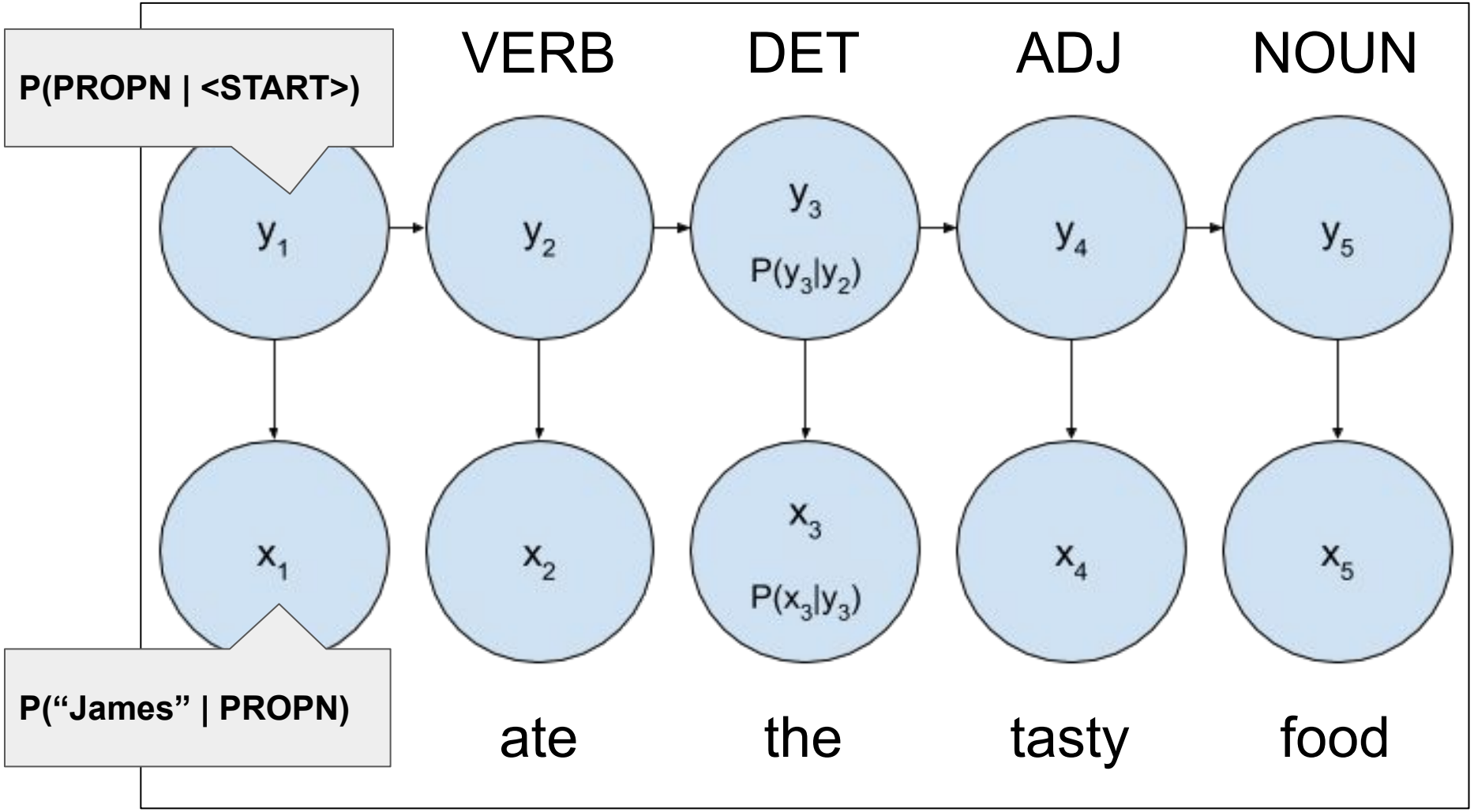
Goal: Predict these!

Observed States: The words in the sequence

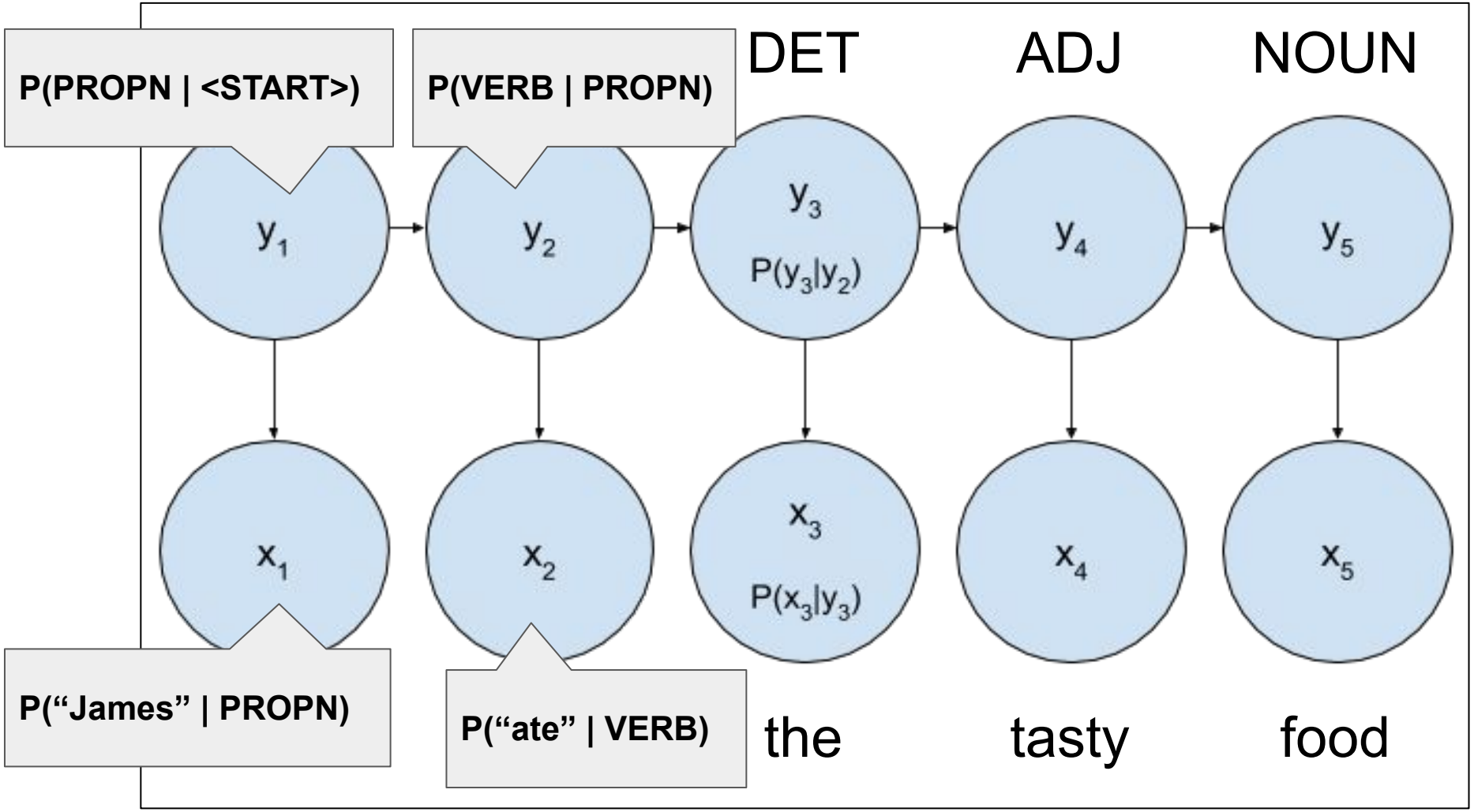
POS tagging with HMMs



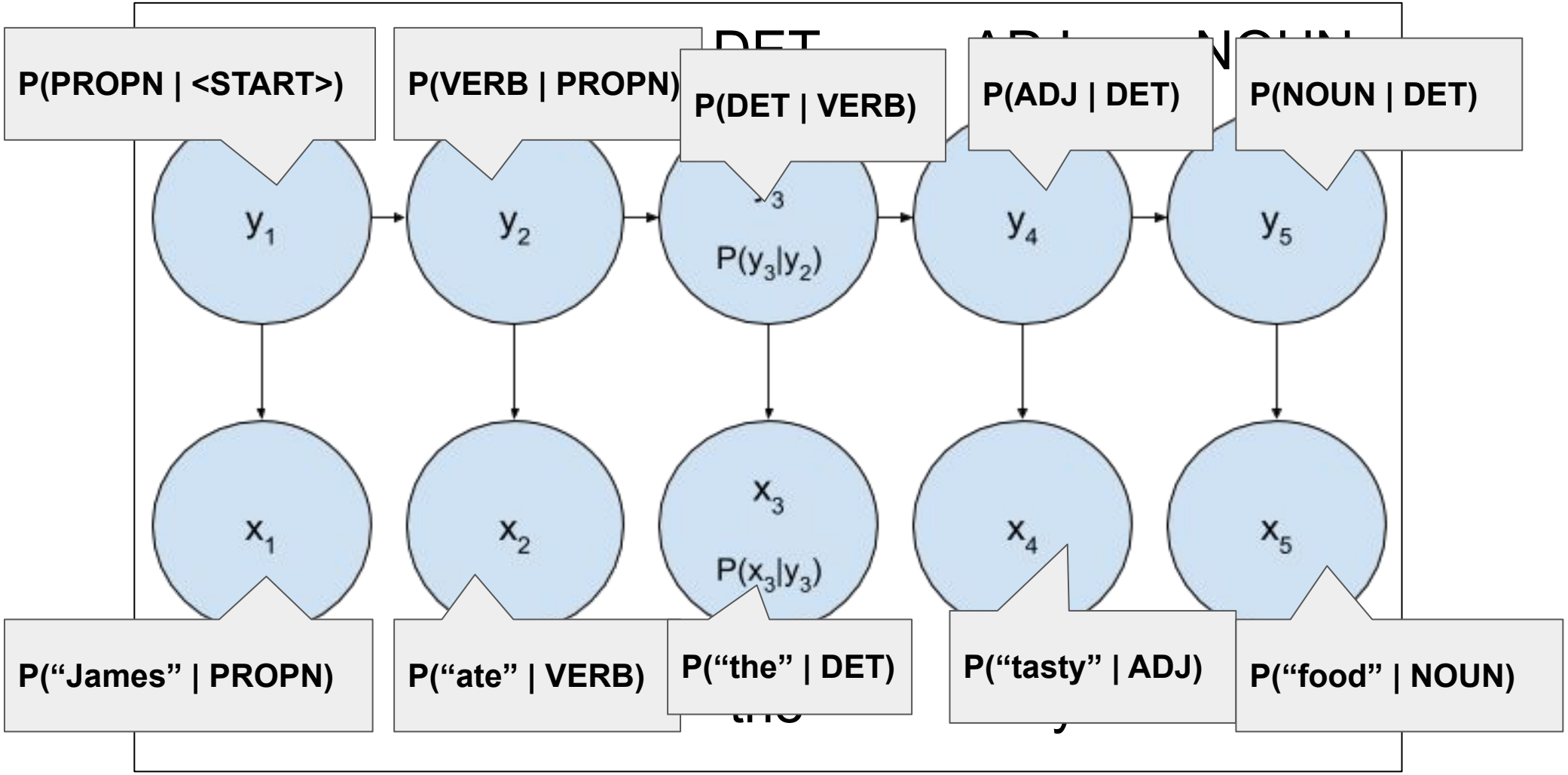
POS tagging with HMMs



POS tagging with HMMs

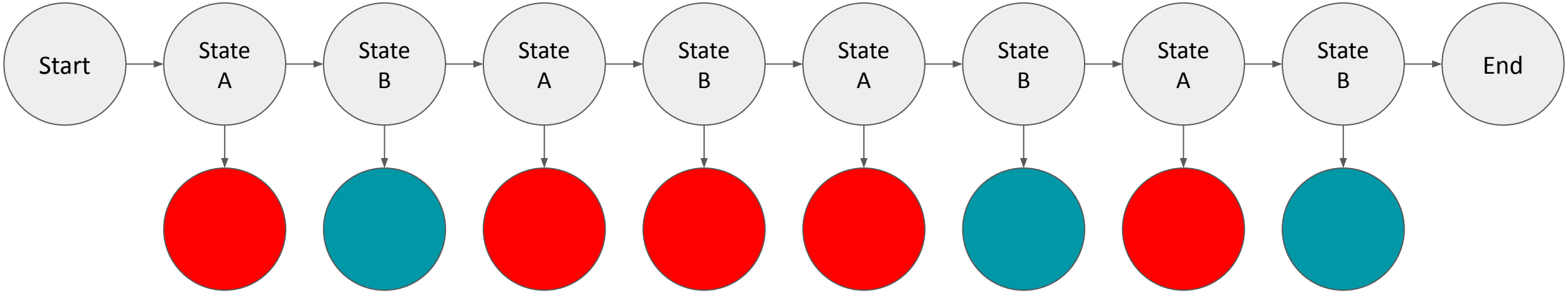


POS tagging with HMMs



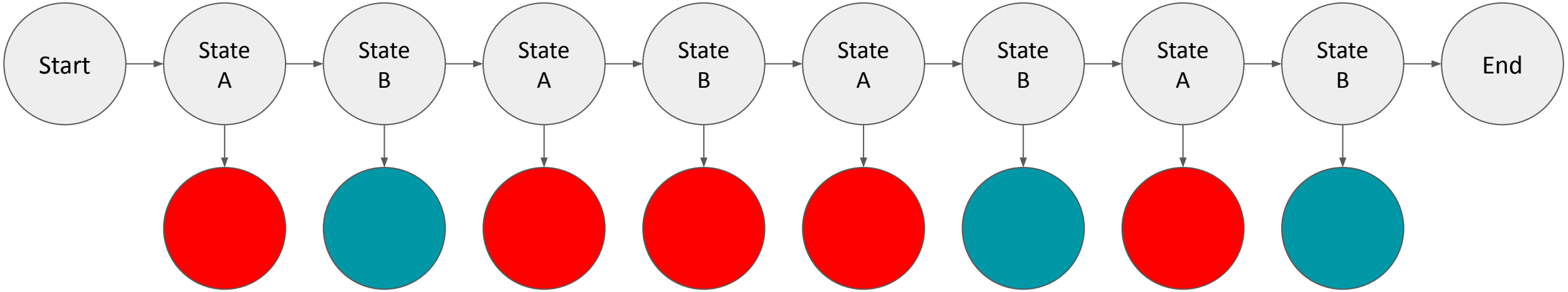
Training time: what can we learn?

- Count and normalise! (*over training data*)



Training time: what can we learn?

- Count and normalise! (*over training data*)

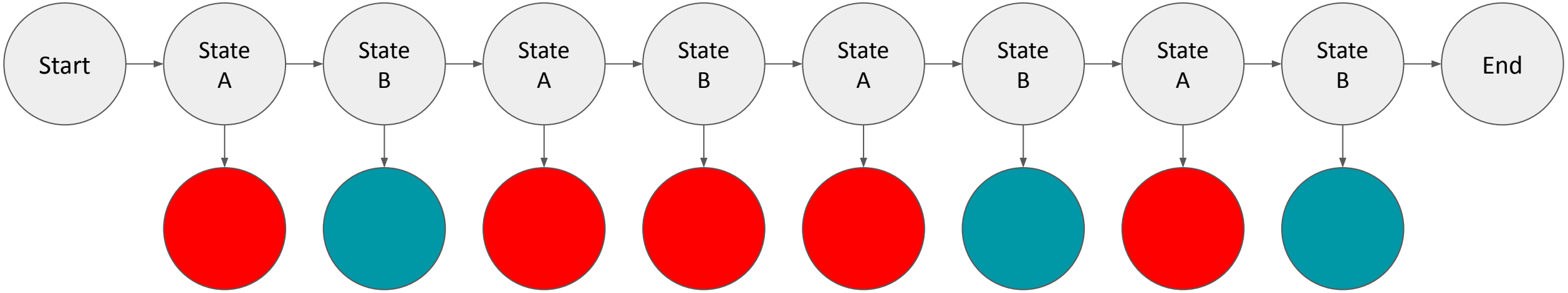


P(Red|A) = 4/4
P(Blue|A) = 0/4
P(Red |B) = 1/4
P(Blue|B) = 3/4

Emission conditional probability table		
	Red	**Blue**
A	1	0
B	1/4	3/4

Training time: what can we learn?

- Count and normalise! (*over training data*)



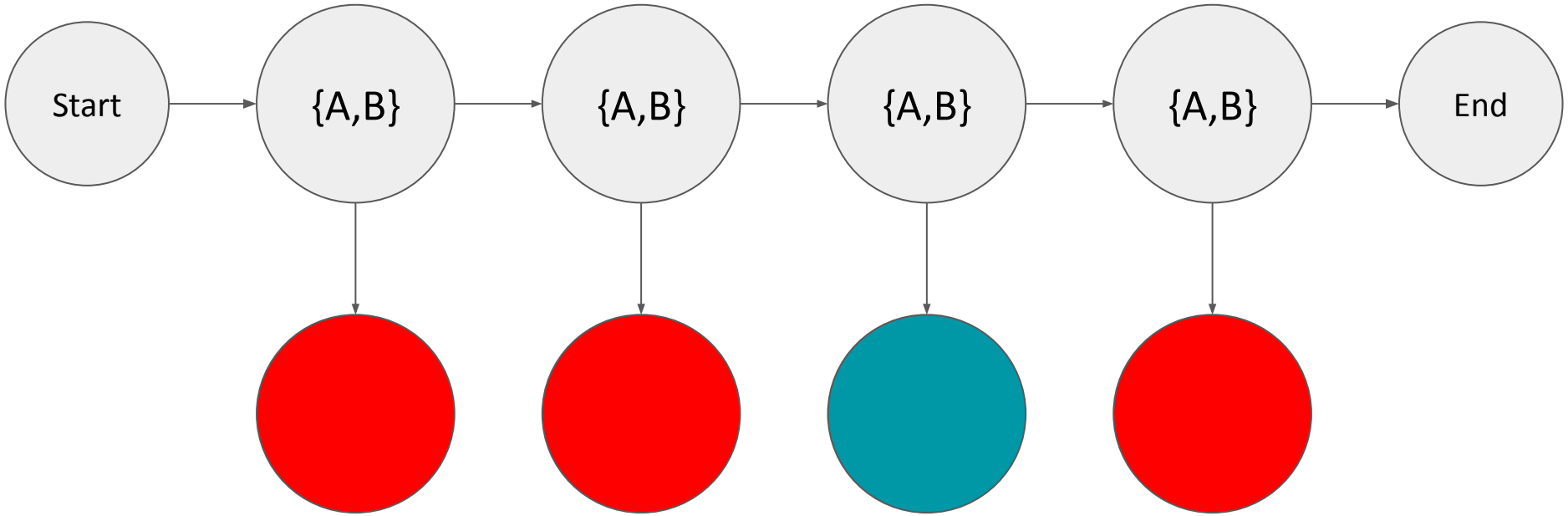
P(Red|A) = 4/4
P(Blue|A) = 0/4
P(Red |B) = 1/4
P(Blue|B) = 3/4

Emission conditional probability table		
	Red	**Blue**
A	1	0
B	1/4	3/4

Transition conditional probability table			
	A	**B**	**<end>**
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4

P(A|s) = 1
P(B|s) = 0
P(A|A) = 0/4 = 0
P(B|A) = 4/4 = 1
P(A|B) = 3/4 = 0.75
P(B|B) = 0/4 = 0
P(END| B) = 1/4 = 0.25
P(END| A) = 0/4 = 0

Inference time: What are the hidden states here?



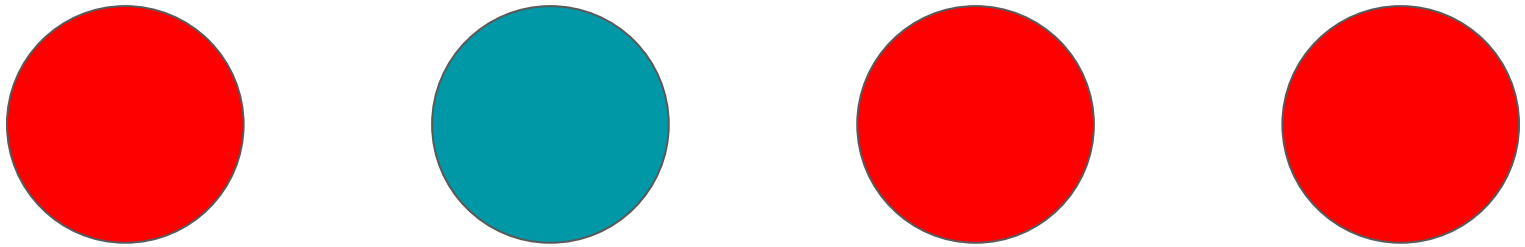
Emission conditional probability table		
	Red	**Blue**
A	1	0
B	1/4	3/4

Transition conditional probability table			
	A	**B**	**<end>**
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4

The Greedy Approach

- Step through the sequence one output at a time (Red, Blue, Red, Red)
- Decide the most likely hidden state given the product of:
  - the transition probability (for that hidden state)
  - the emission probability (of that hidden state outputting the given output)

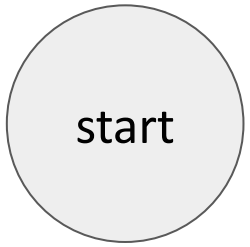
Greedy because it's picking the most likely choice at each outcome and not looking ahead at all

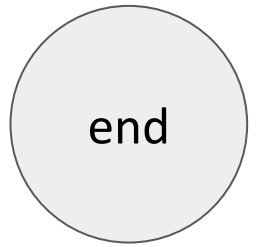


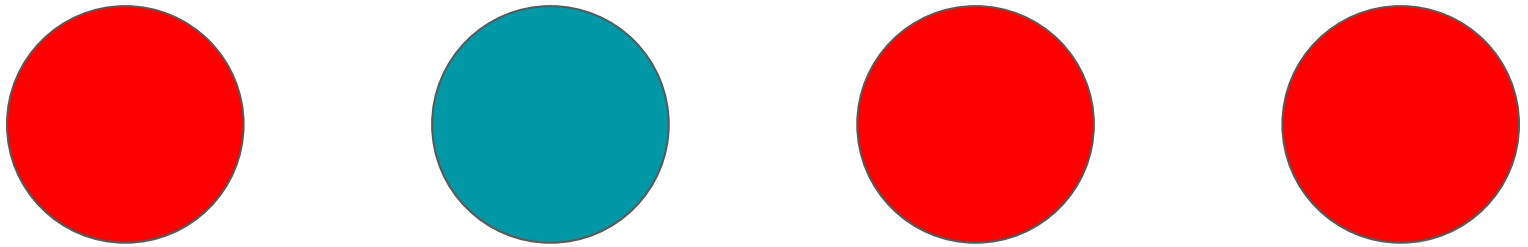
Let's calculate some probabilities!

Emission table		
	Red	Blue
A	1	0
B	1/4	3/4

Transition table			
	A	B	<end>
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4



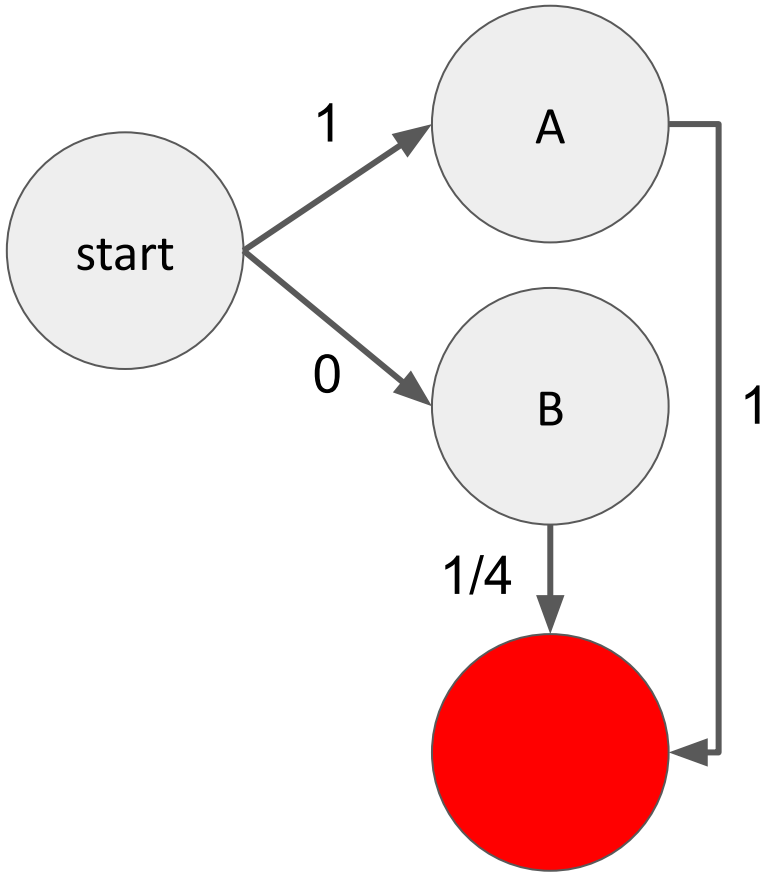


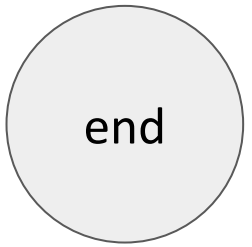


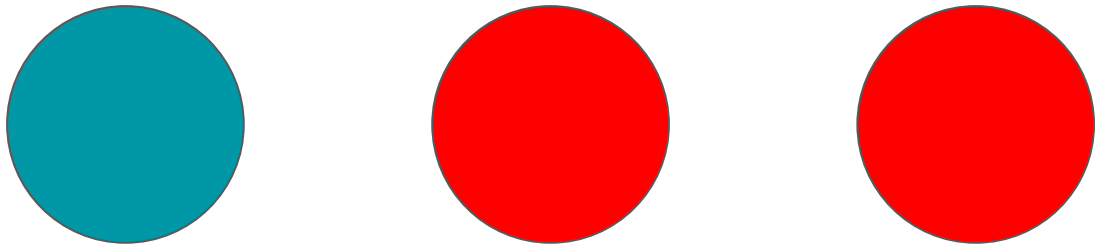
P(A|S,RED) = 1 * 1= 1
P(B|S,RED) = 0 * 1/4 = 0

Emission table		
	Red	**Blue**
A	1	0
B	1/4	3/4

Transition table			
	A	**B**	**<end>**
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4



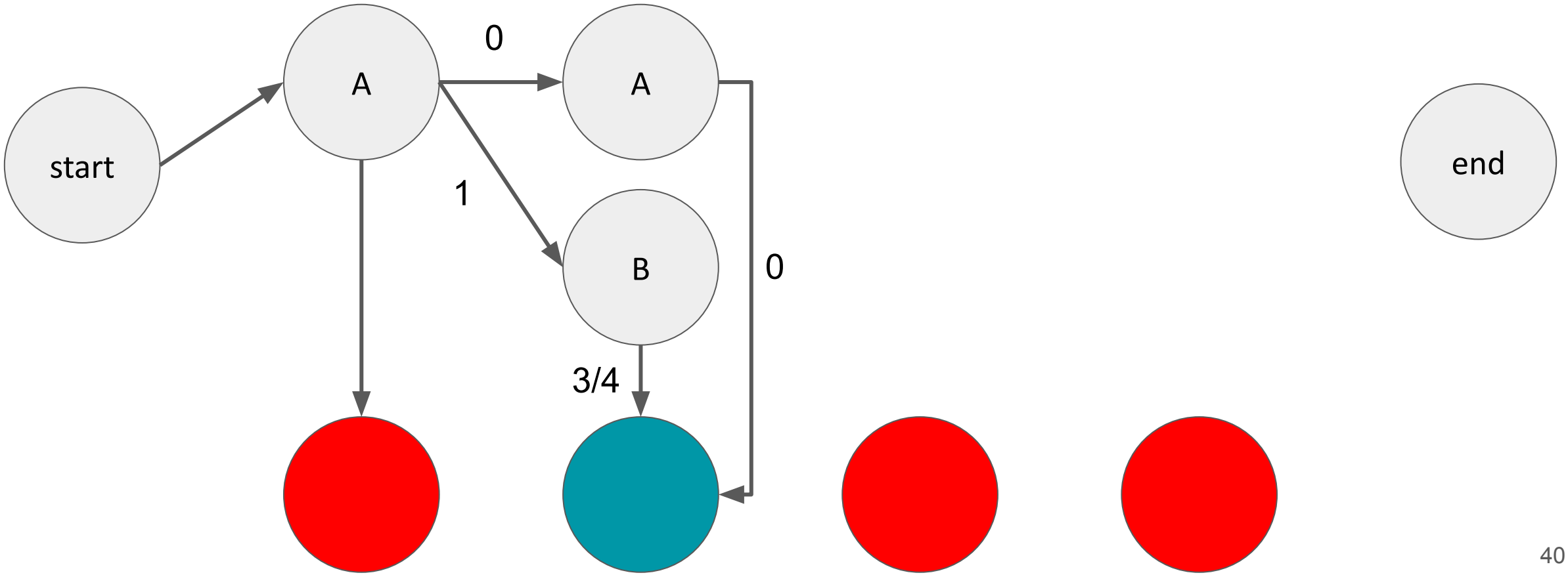




Emission table		
	Red	Blue
A	1	0
B	1/4	3/4

Transition table			
	A	B	<end>
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4

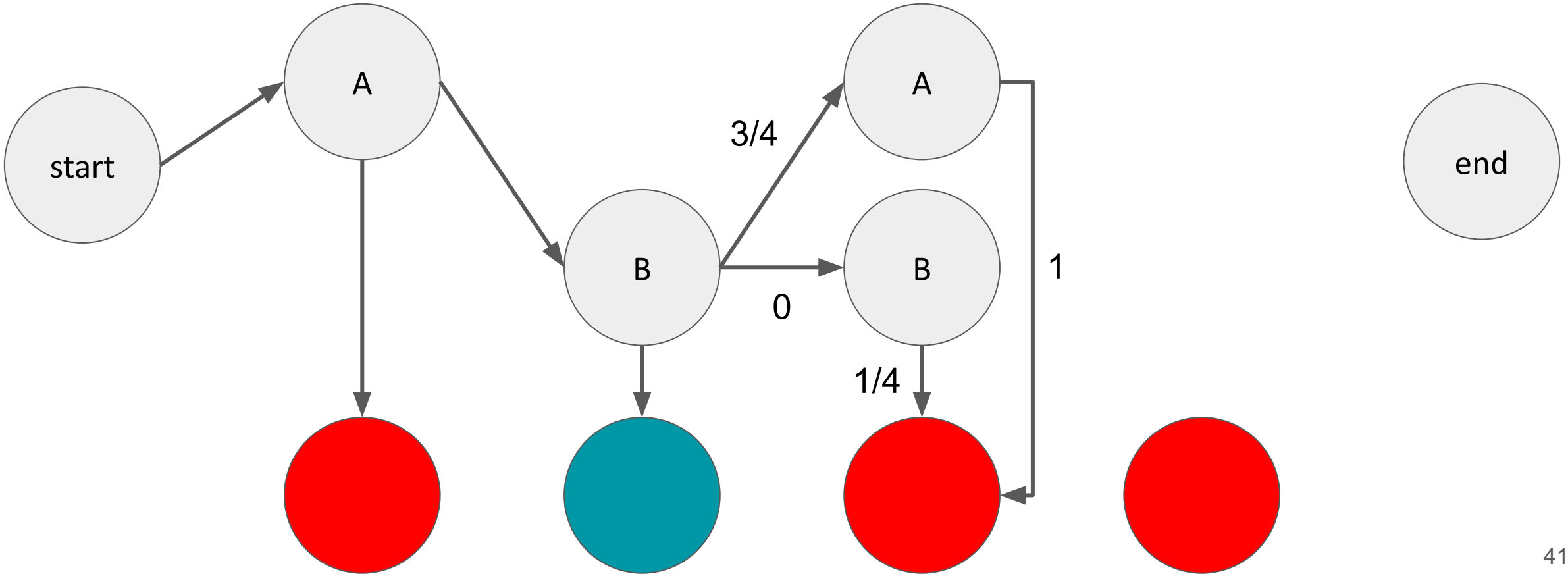
P(A|A,BLUE) = 0 * 0 = 0
P(B|A,BLUE) = 1 * 3/4 = 3/4



P(A|B,RED) = 3/4 * 1 = 3/4
P(B|B,RED) = 0 * 1/4 = 0

Emission table		
	Red	**Blue**
A	1	0
B	1/4	3/4

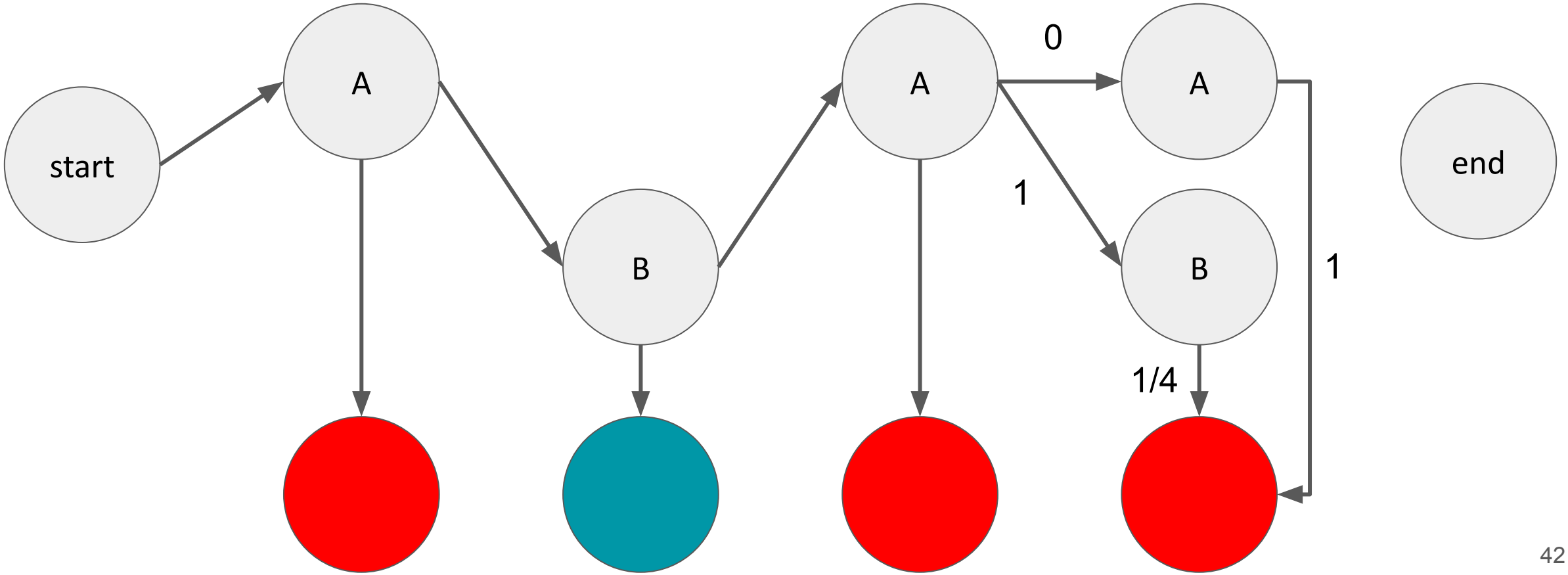
Transition table			
	A	**B**	**<end>**
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4



P(A|A,RED) = 0 * 1= 0
P(B|A,RED) = 1 * 1/4 = 1/4

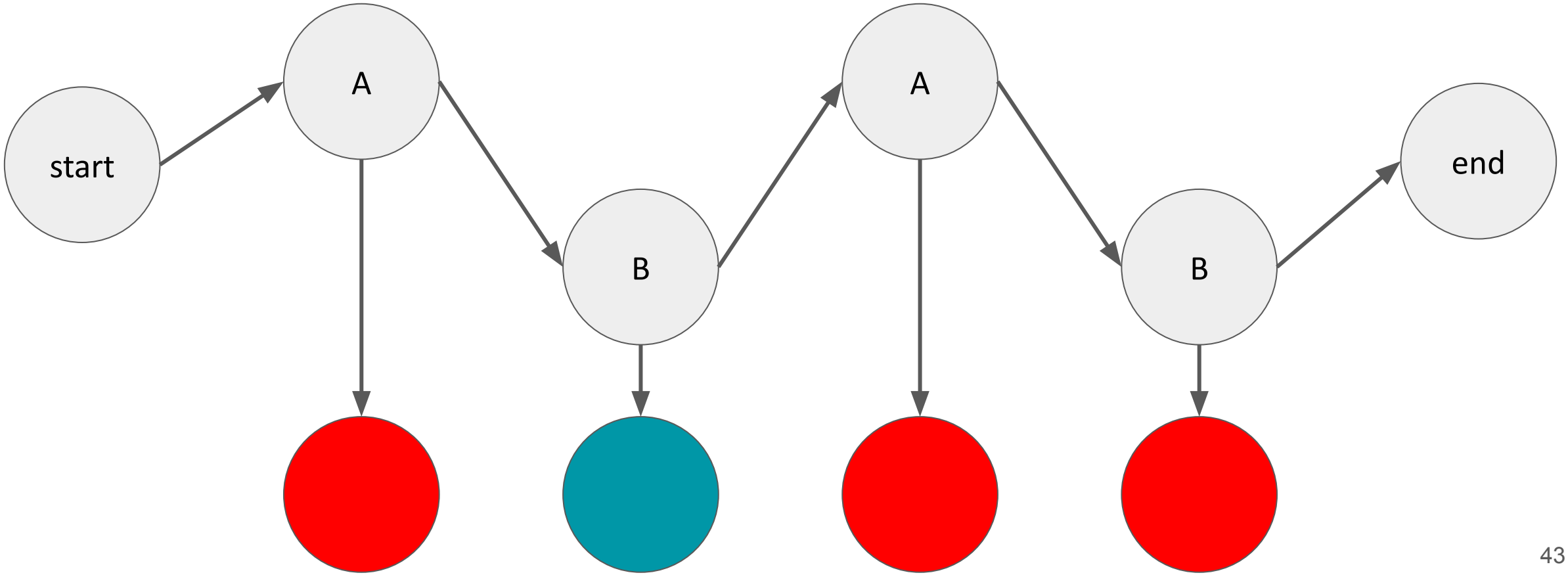
Emission table		
	Red	**Blue**
A	1	0
B	1/4	3/4

Transition table			
	A	**B**	**<end>**
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4



Emission table		
	Red	Blue
A	1	0
B	1/4	3/4

Transition table			
	A	B	<end>
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4



HMMs: Part-of-speech (POS) Training

Transitions: $p(y_i \mid y_{i-1})$

P(V | N) = 1/2

P(DT | V) = 1/1

P(N | DT) = 1/1

P(END | N) = 1/2

Emissions: $p(x_i \mid y_i)$

P(James | N) = 1/2

P(ate | V) = 1

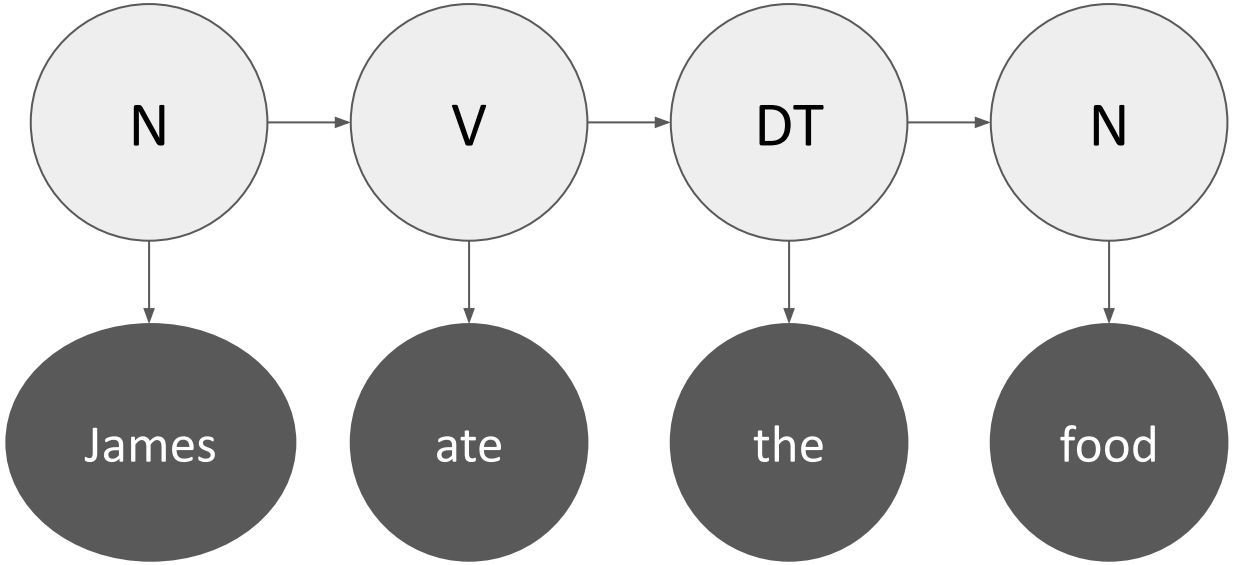
P(the | DT) = 1

P(food | N) = 1/2

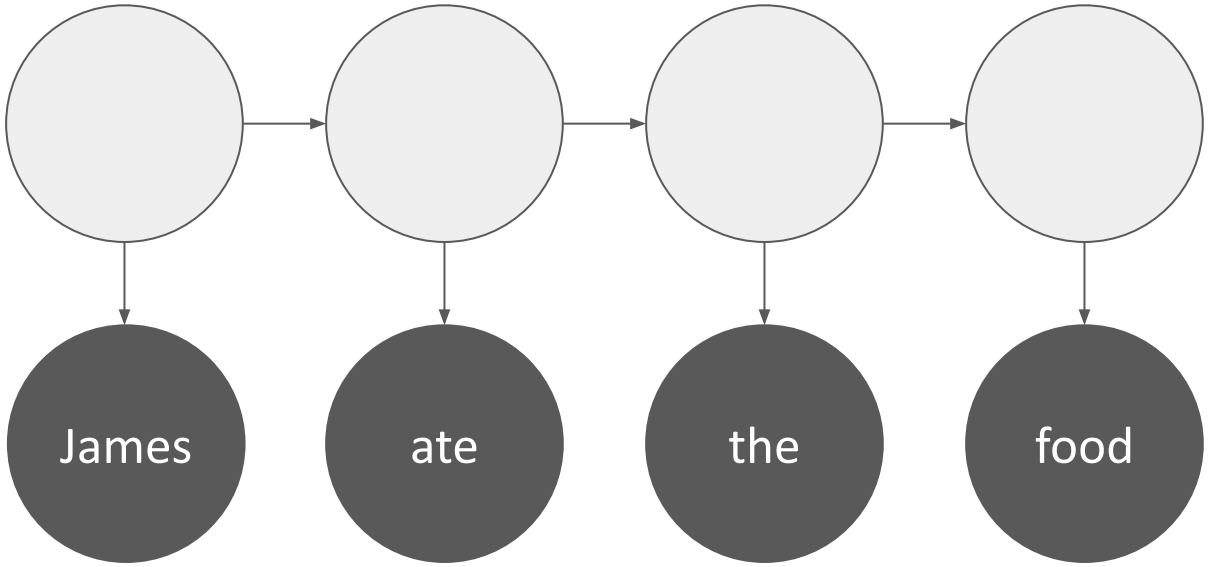
The hidden states are parts-of-speech!

Count and normalise! (*over training data*)

- From manually labelled text data

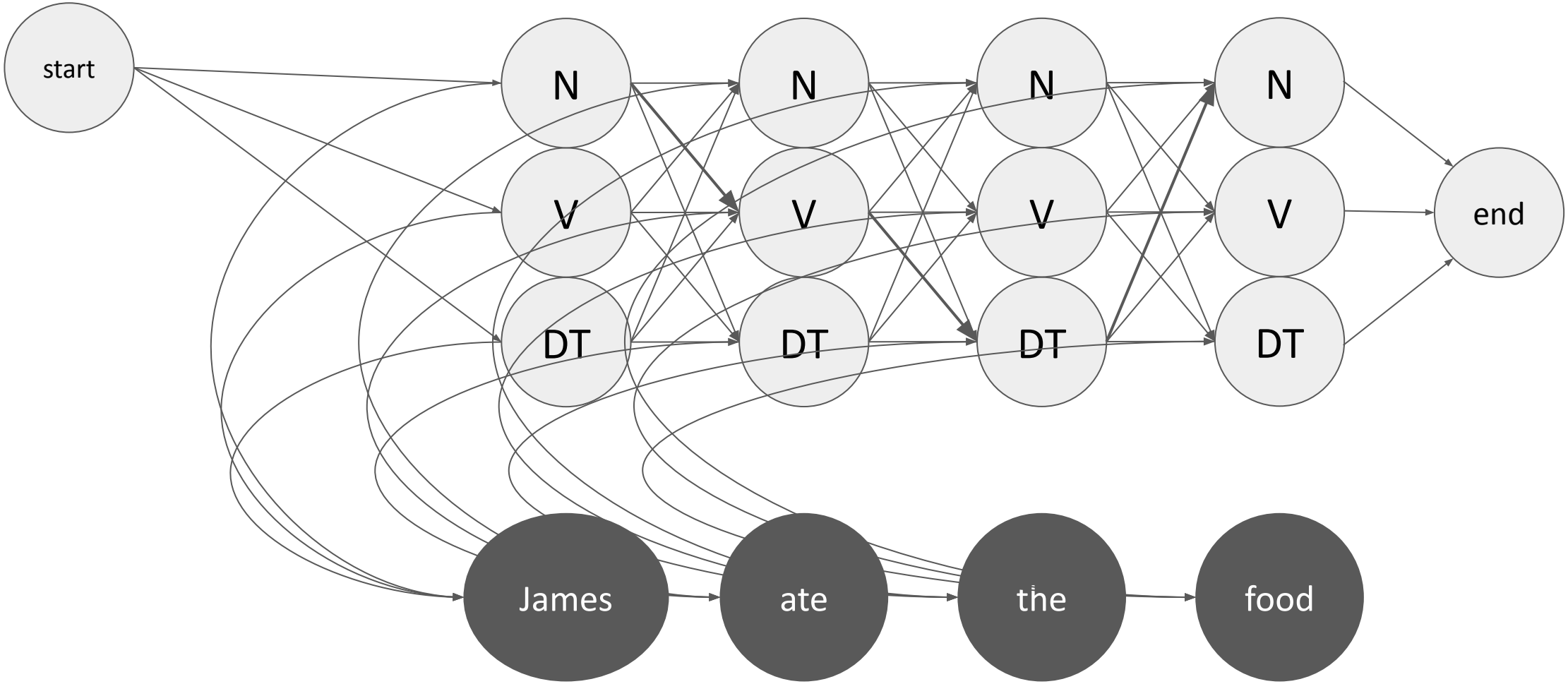


HMMs: POS Inference



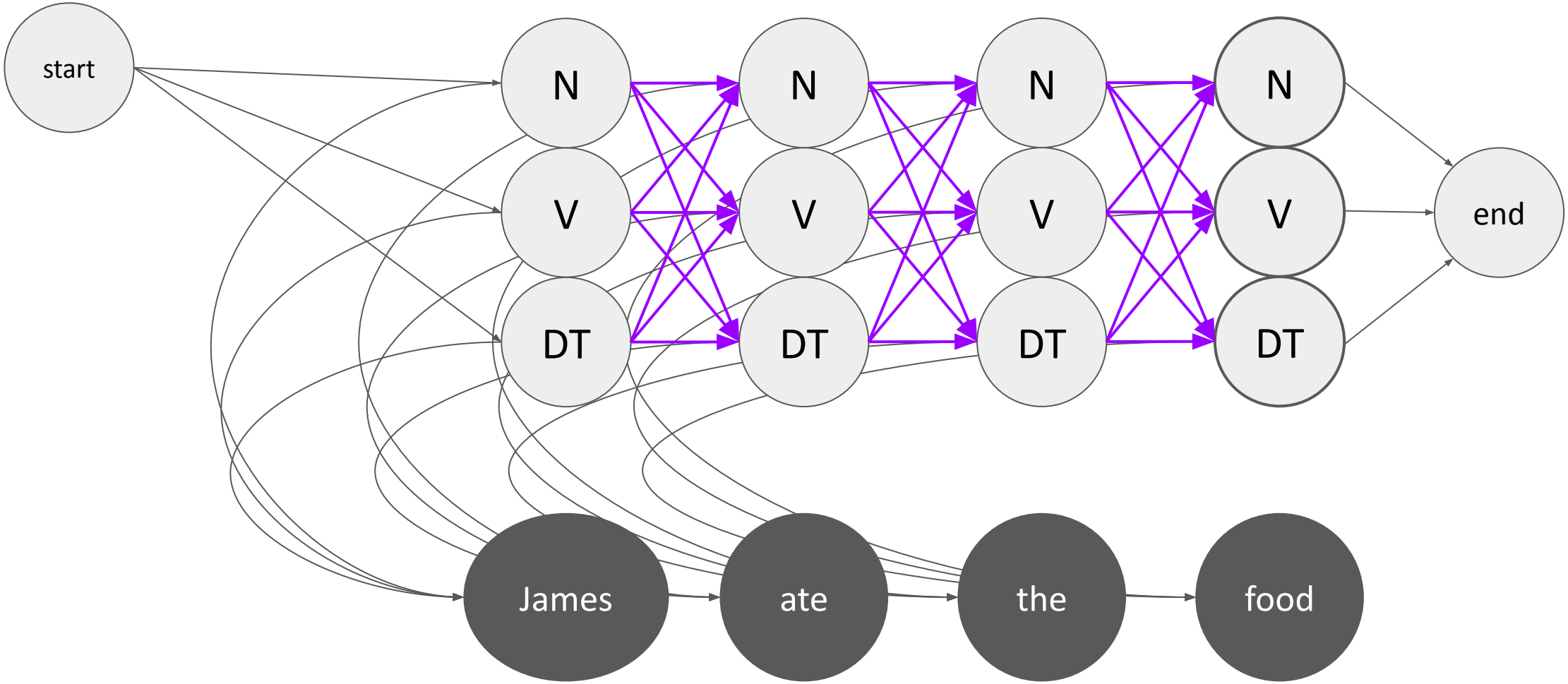
Finding the hidden states gives us parts-of-speech tags for each token

HMMs: POS Inference

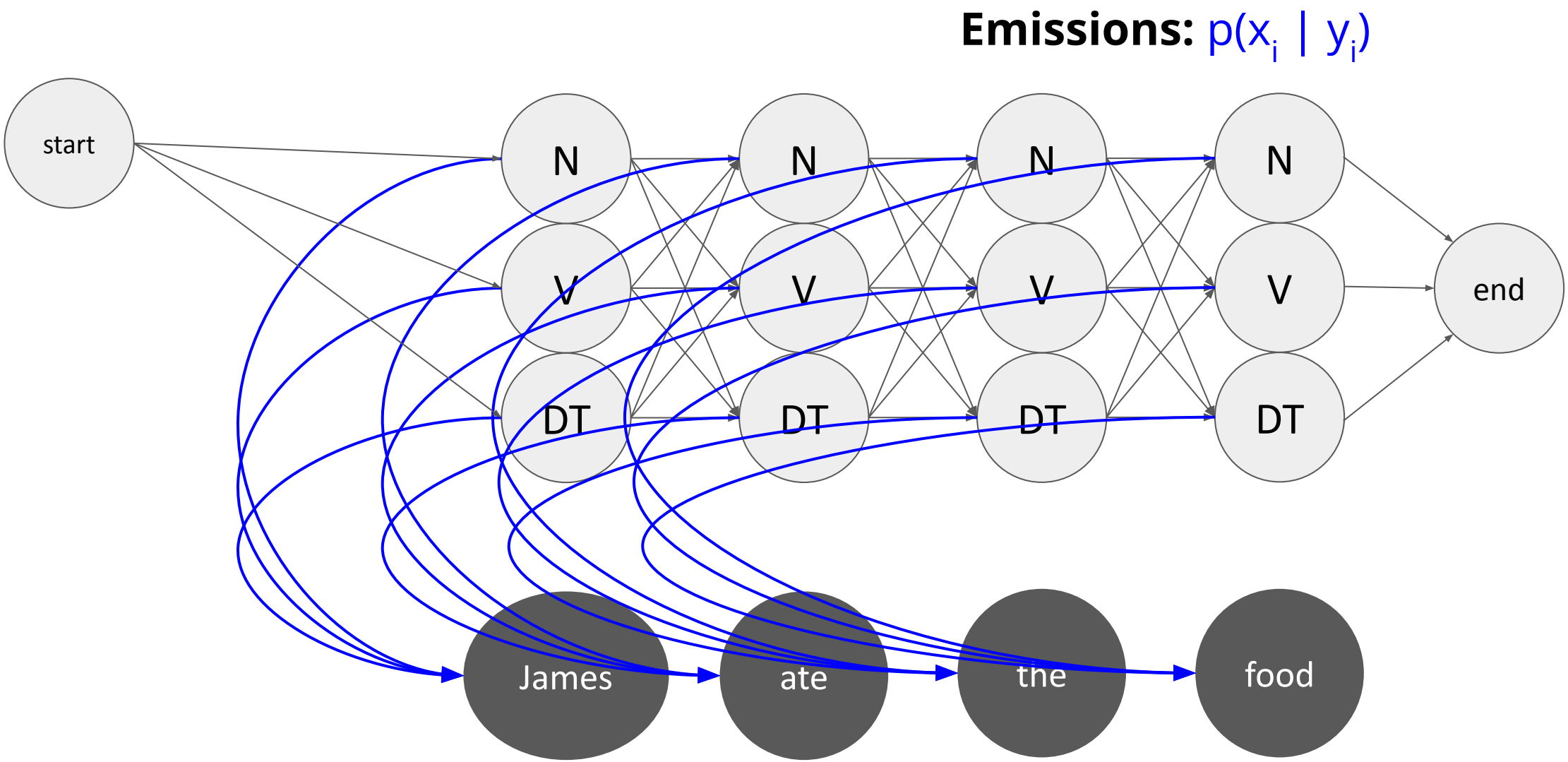


HMMs: POS Inference

Transitions: $p(y_i \mid y_{i-1})$



HMMs: POS Inference



Exercise: HMM Inference

Calculate the probabilities and find the optimal sequence for the sentence using a greedy search.

“Sue drew Mark”

Transitions: $P(y_i \mid y_{i-1})$

P(VERB | <START>) = 1/4
P(PROPN | <START>) = 3/4

P(VERB | VERB) = 0
P(PROPN | VERB) = 1

P(VERB | PROPN) = 2/4
P(PROPN | PROPN) = 2/4

Emissions: $P(x_i \mid y_i)$

P(sue | PROPN) = 1/6
P(drew | PROPN) = 2/6
P(mark | PROPN) = 3/6

P(sue | VERB) = 2/6
P(drew | VERB) = 3/6
P(mark | VERB) = 2/6

Exercise: HMM Inference

"Sue drew Mark"

P(VERB | <START>) * P(sue | VERB) = 1/4 * 2/6 = 2/24

P(PROPN | <START>) * P(sue | PROPN) = 3/4 * 1/6 = 3/24

P(VERB | PROPN) * P(drew | VERB) = 2/4 * 3/6 = 6/24

P(PROPN | PROPN) * P(drew | PROPN) = 2/4 * 2/6 = 4/24

P(VERB | VERB) * P(mark | VERB) = 0 * 2/6 = 0

P(PROPN | VERB) * P(mark | PROPN) = 1 * 3/6 = 3/6

Assignment: PROPN, VERB, PROPN

Transitions: $P(y_i \mid y_{i-1})$

P(VERB | <START>) = 1/4
P(PROPN | <START>) = 3/4

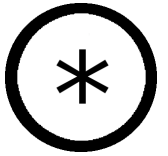
P(VERB | VERB) = 0
P(PROPN | VERB) = 1

P(VERB | PROPN) = 2/4
P(PROPN | PROPN) = 2/4

Emissions: $P(x_i \mid y_i)$

P(sue | PROPN) = 1/6
P(drew | PROPN) = 2/6
P(mark | PROPN) = 3/6

P(sue | VERB) = 2/6
P(drew | VERB) = 3/6
P(mark | VERB) = 2/6



Break!

Up next: the Viterbi algorithm

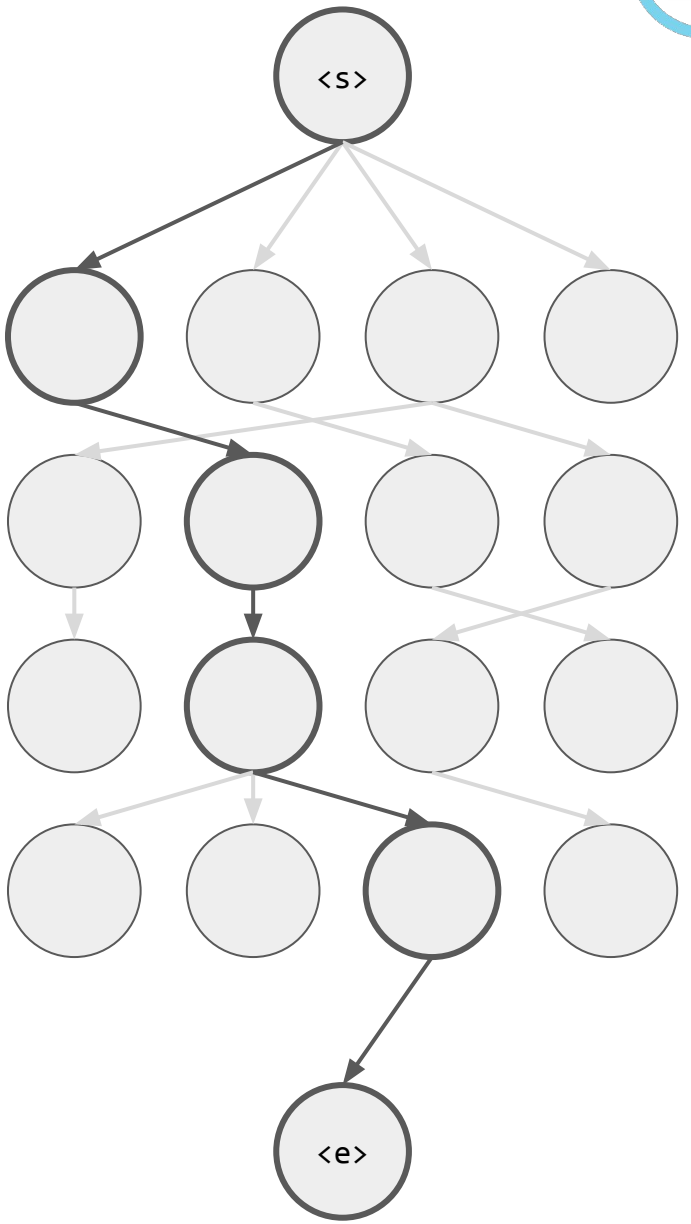
The Viterbi Algorithm

Superior to the greedy method

- Takes the whole sequence into account when deciding the most likely tags

Basic idea:

- Calculate the maximum possible probability of the entire sequence given transition and emission probabilities
- Be clever about it and build the probabilities up one token at a time (using dynamic programming)
- Then work backwards to find what the optimal path through the hidden states was
  - This gives you the POS tags



Viterbi is an example of Dynamic Programming

- Dynamic programming solves problems that rely on recursion more efficiently
- Efficiently builds up the solution

Example Problem: Calculate the 100th Fibonacci multiplied by the 101st Fibonacci

- Doing `fib(100)` and `fib(101)` independently would repeat calculations
- So build up `fib(1)` to `fib(101)` and store/use the results as you go

```
fib(n) = fib(n-1) + fib(n-2)
```

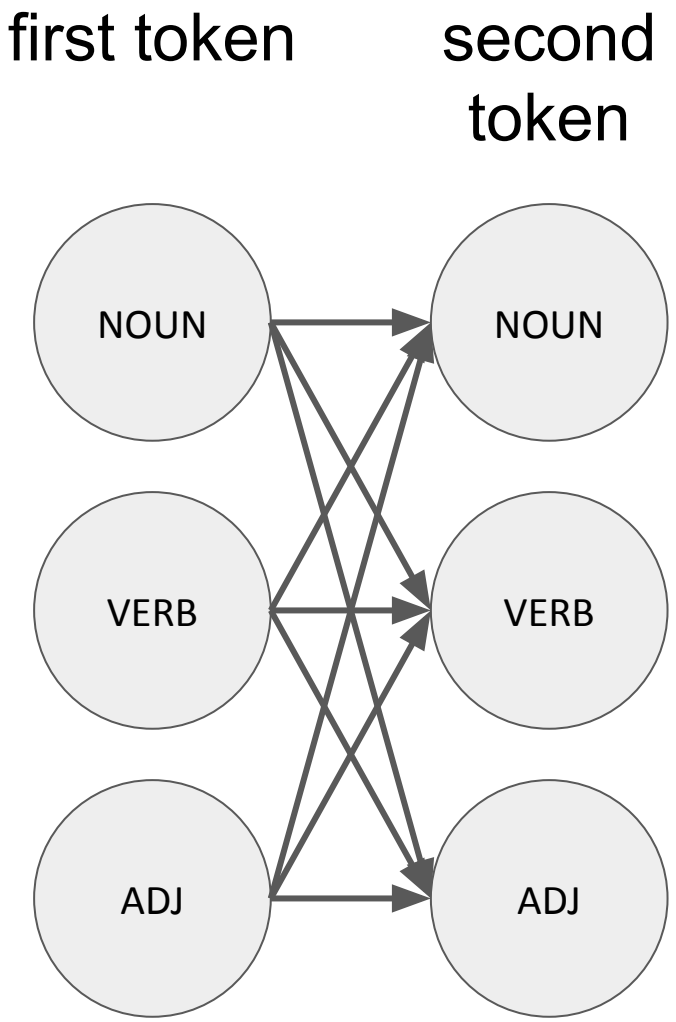
created with Copilot and DALL-E 3

The Viterbi Algorithm

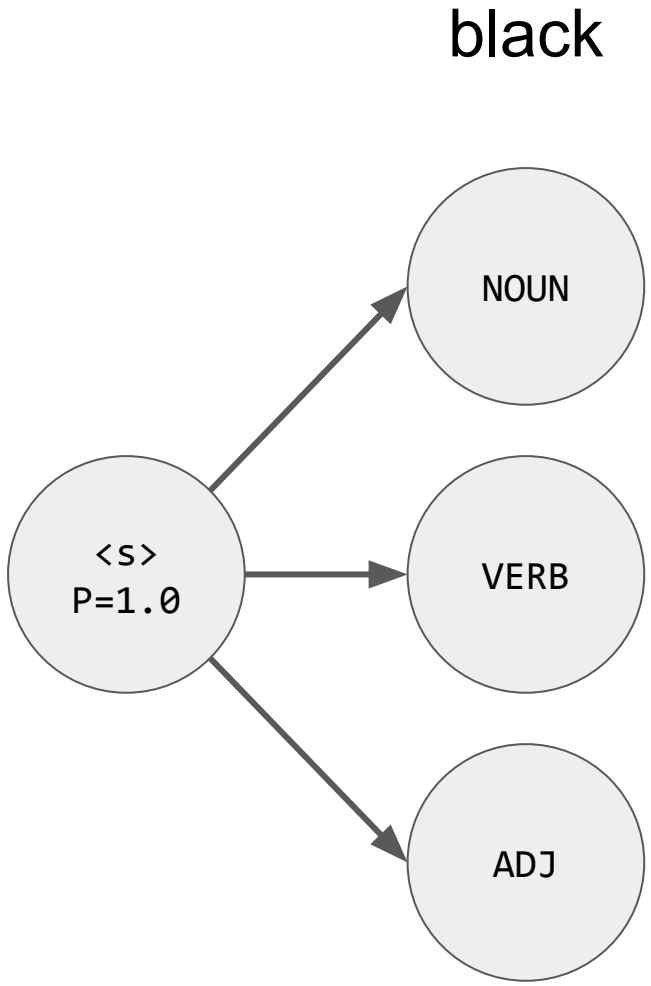
Algorithm:

Work through the sequence one output/emission at a time

- Use the probabilities of reaching each of the previous hidden states
- Find the most likely transition and emission for this output/emission
  - Decide which is the most likely previous state at each step and use that

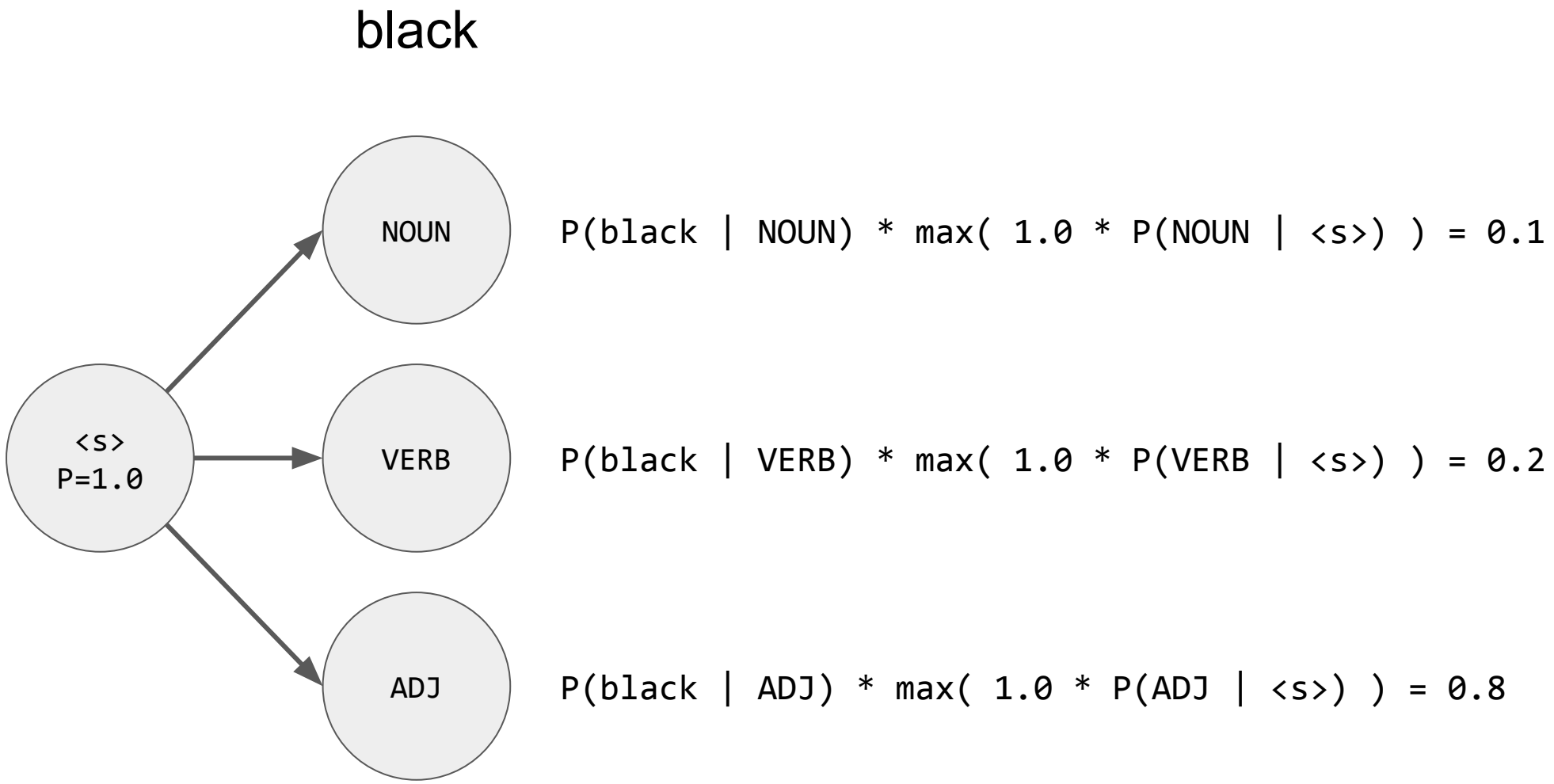


Viterbi Demo for “black bears go home”



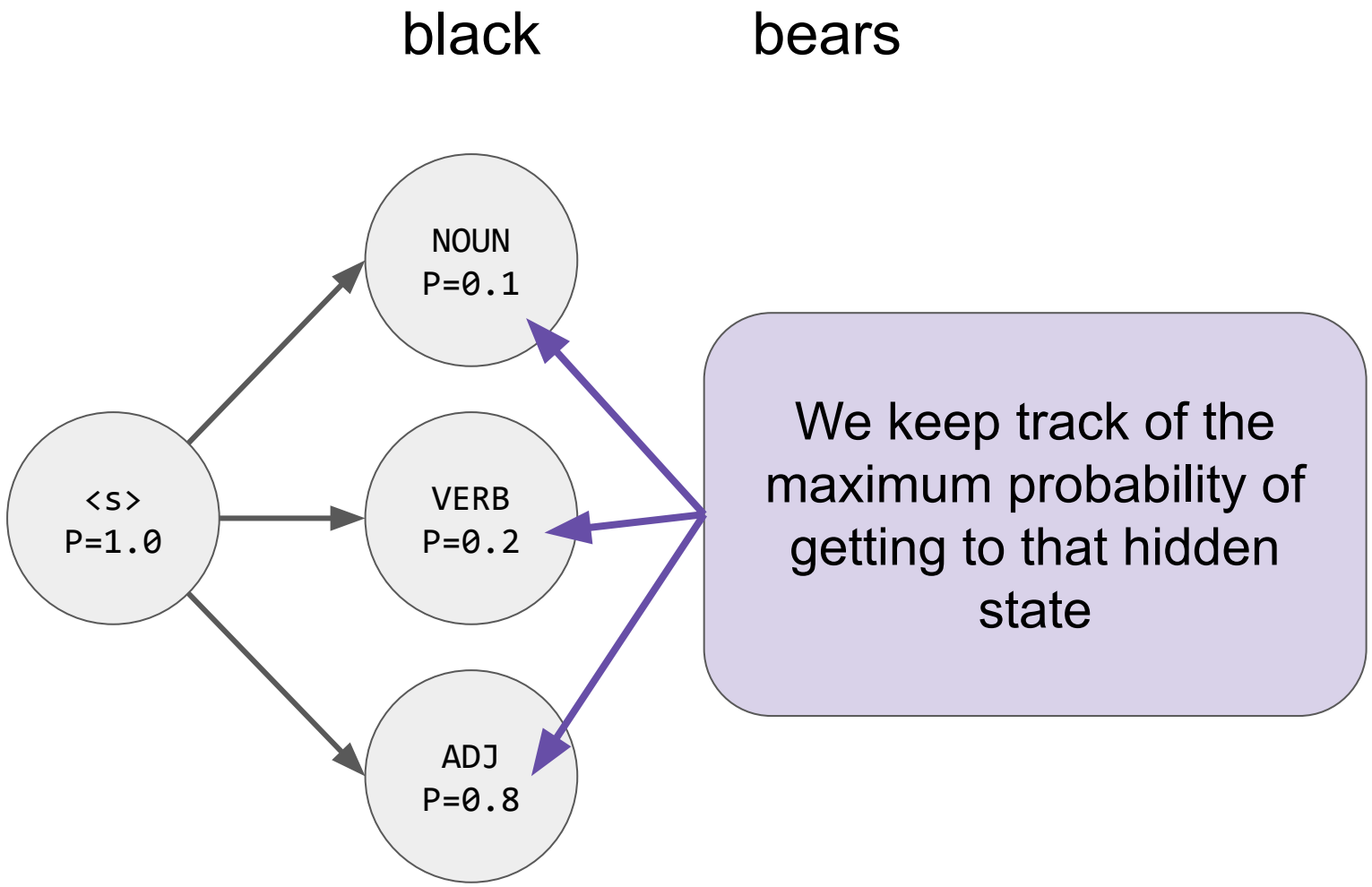
Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”



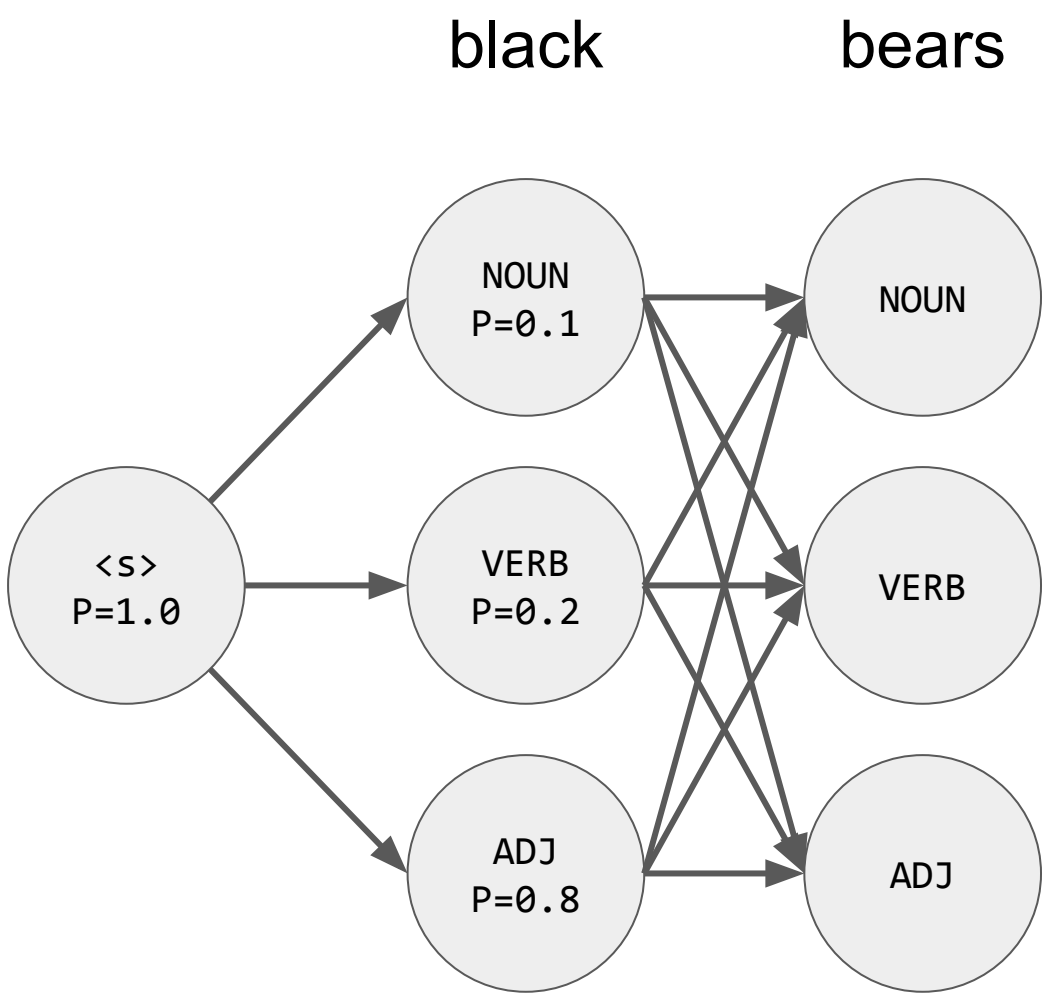
Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”



Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”



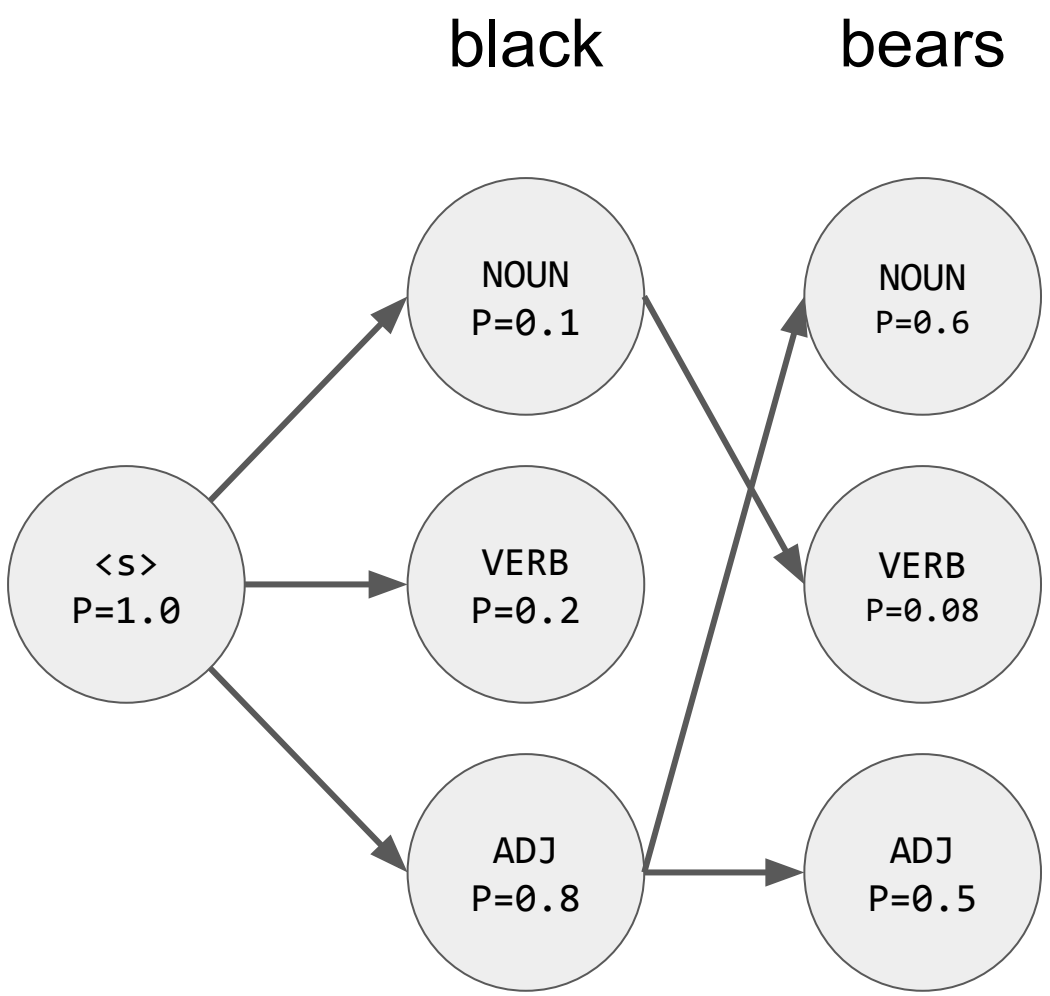

```
                                    0.1 * P(NOUN | NOUN),
P(bears | NOUN) * max( 0.2 * P(NOUN | VERB), )
                                    0.8 * P(NOUN | ADJ)

                                    0.1 * P(VERB | NOUN),
P(bears | VERB) * max( 0.2 * P(VERB | VERB), )
                                    0.8 * P(VERB | ADJ)

                                    0.1 * P(ADJ | NOUN),
P(bears | ADJ ) * max( 0.2 * P(ADJ | VERB), )
                                    0.8 * P(ADJ | ADJ)
```

Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”



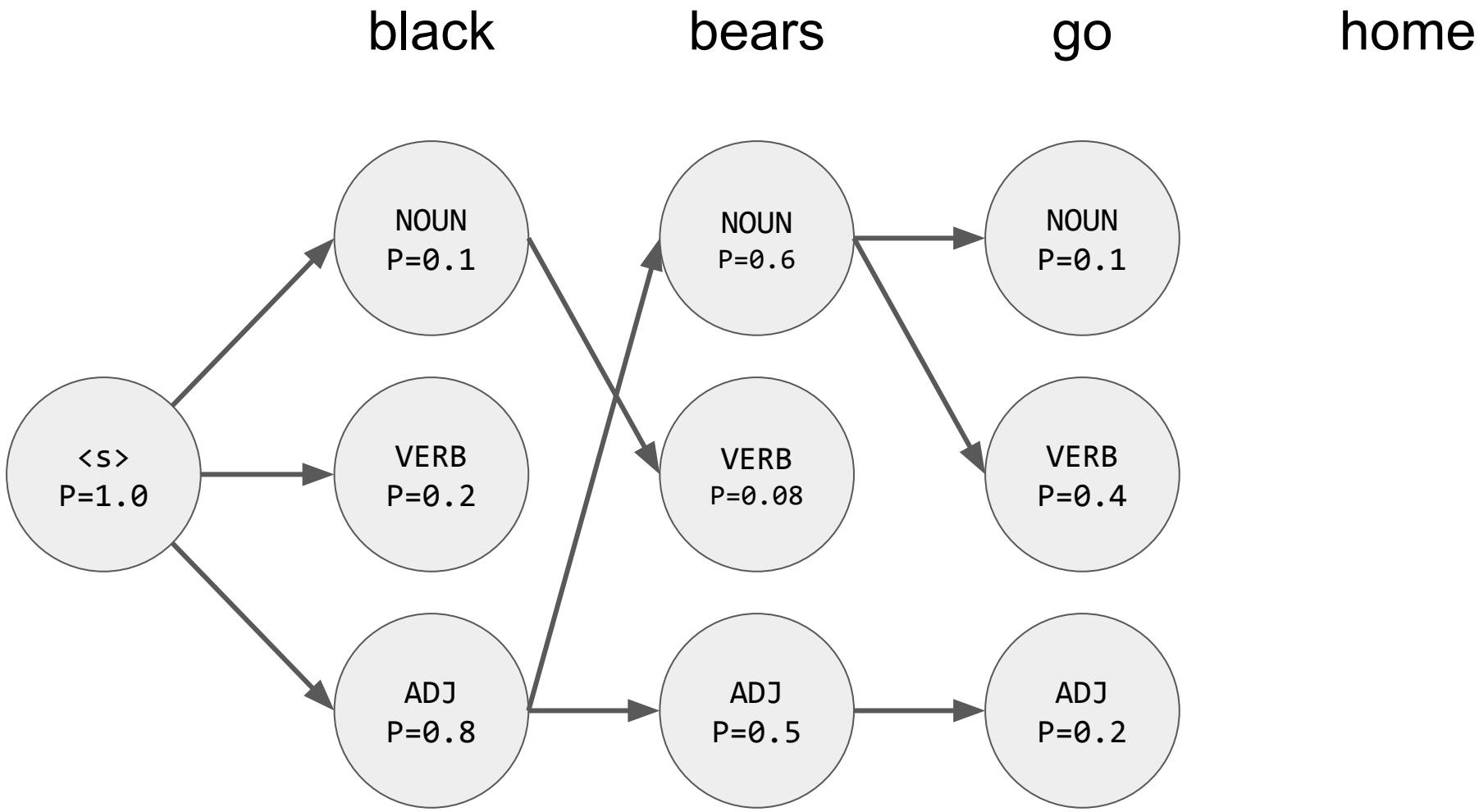

```
P(bears | NOUN) * max( 0.1 * P(NOUN | NOUN),
                       0.2 * P(NOUN | VERB), )
                       0.8 * P(NOUN | ADJ)

P(bears | VERB) * max( 0.1 * P(VERB | NOUN),
                       0.2 * P(VERB | VERB), )
                       0.8 * P(VERB | ADJ)

P(bears | ADJ ) * max( 0.1 * P(ADJ | NOUN),
                       0.2 * P(ADJ | VERB), )
                       0.8 * P(ADJ | ADJ)
```

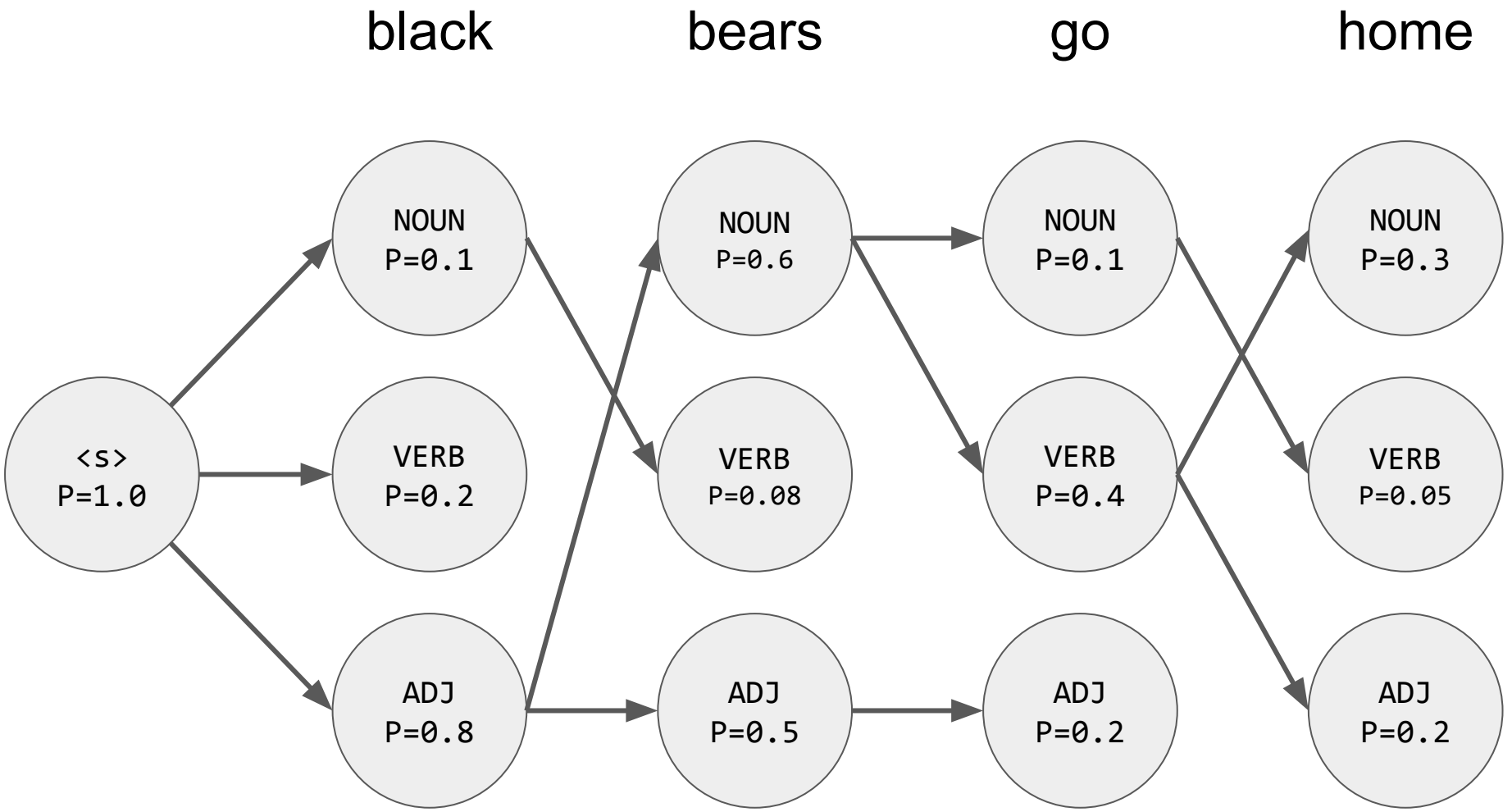
Choose the most likely transition to each hidden state

Viterbi Demo for “black bears go home”



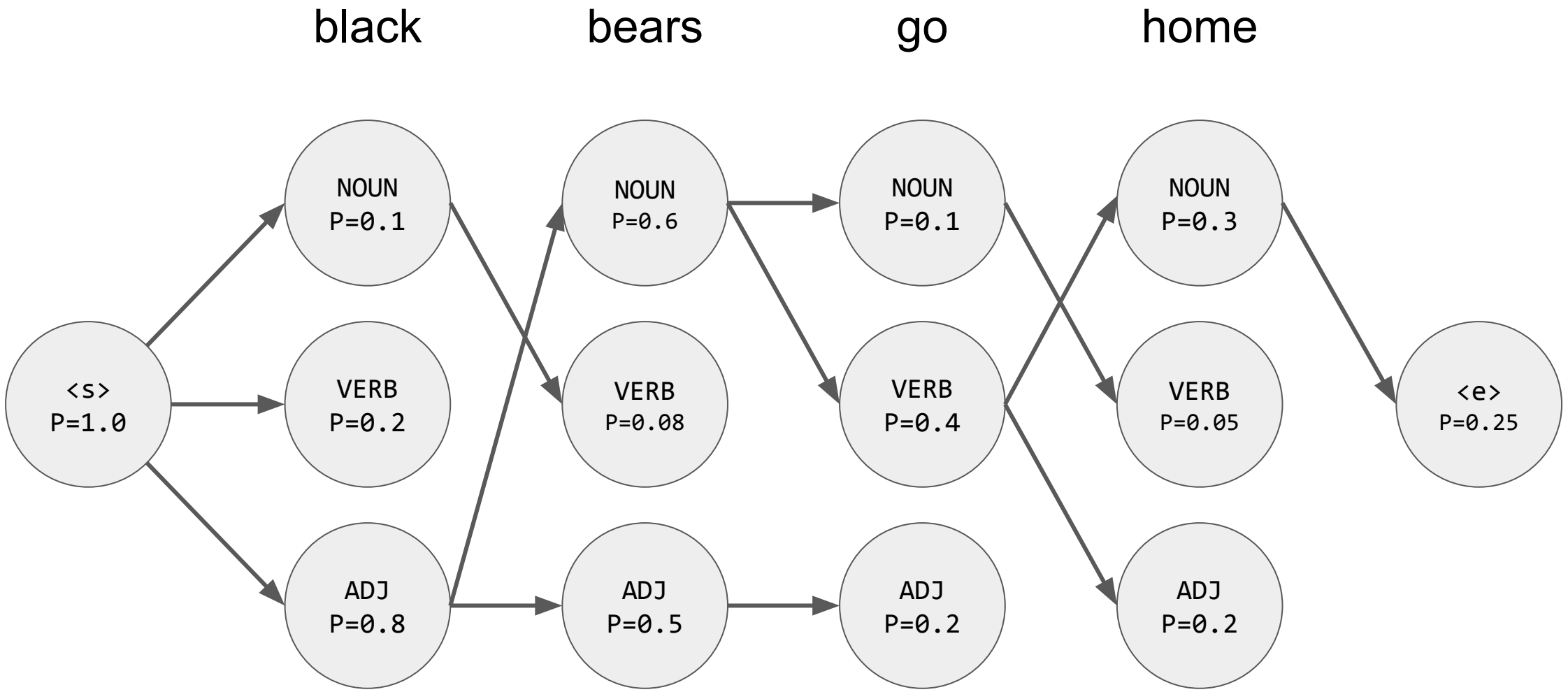
Repeat: Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”



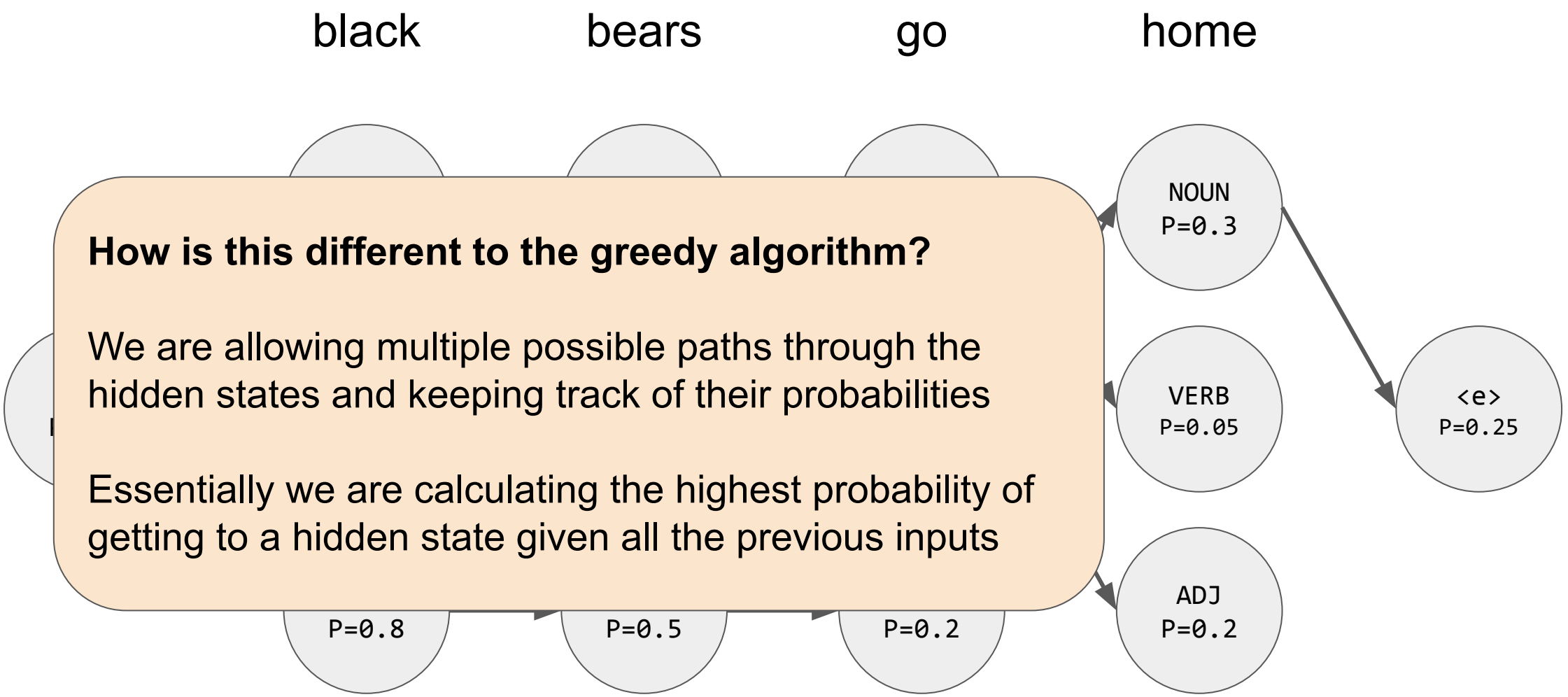
Repeat: Find the probability of each hidden state given the probabilities of previous states

Viterbi Demo for “black bears go home”

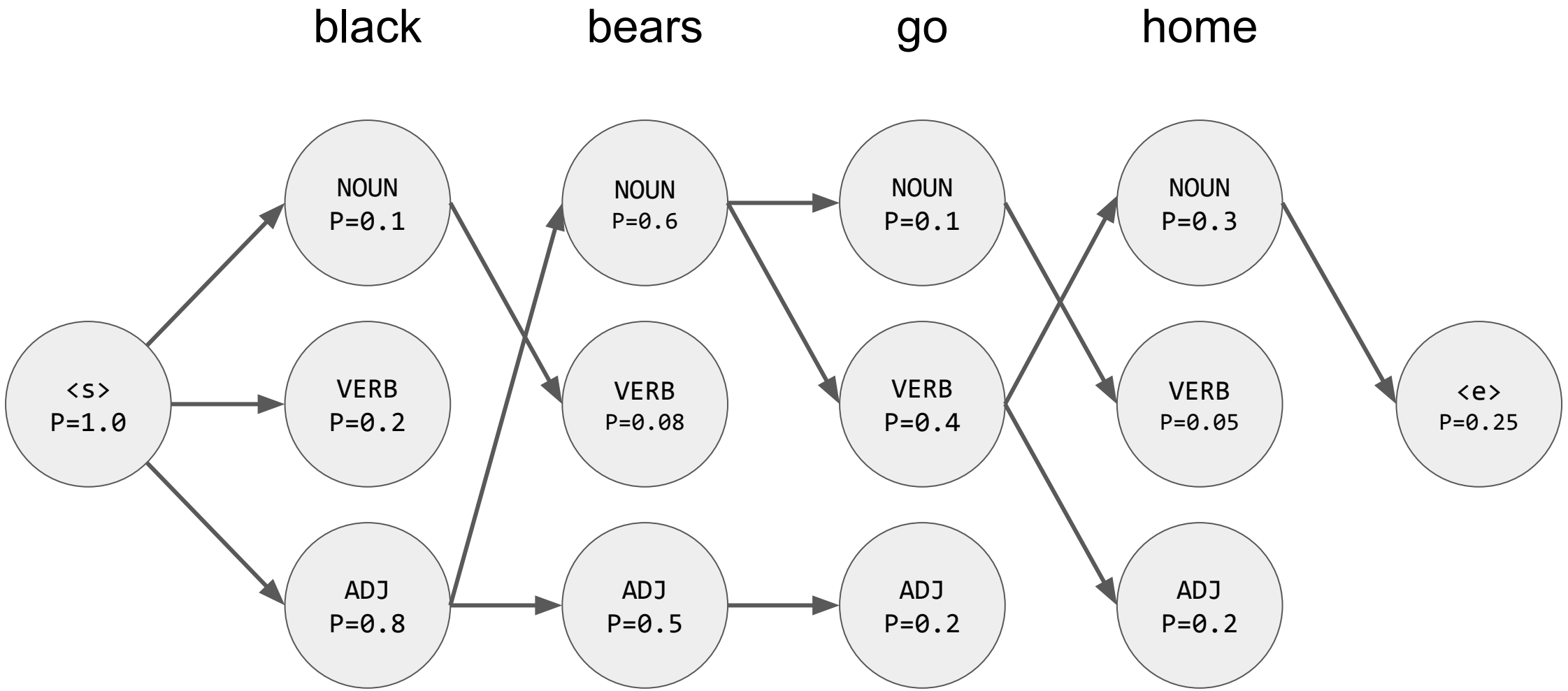


Final step: Find last step to <end> state

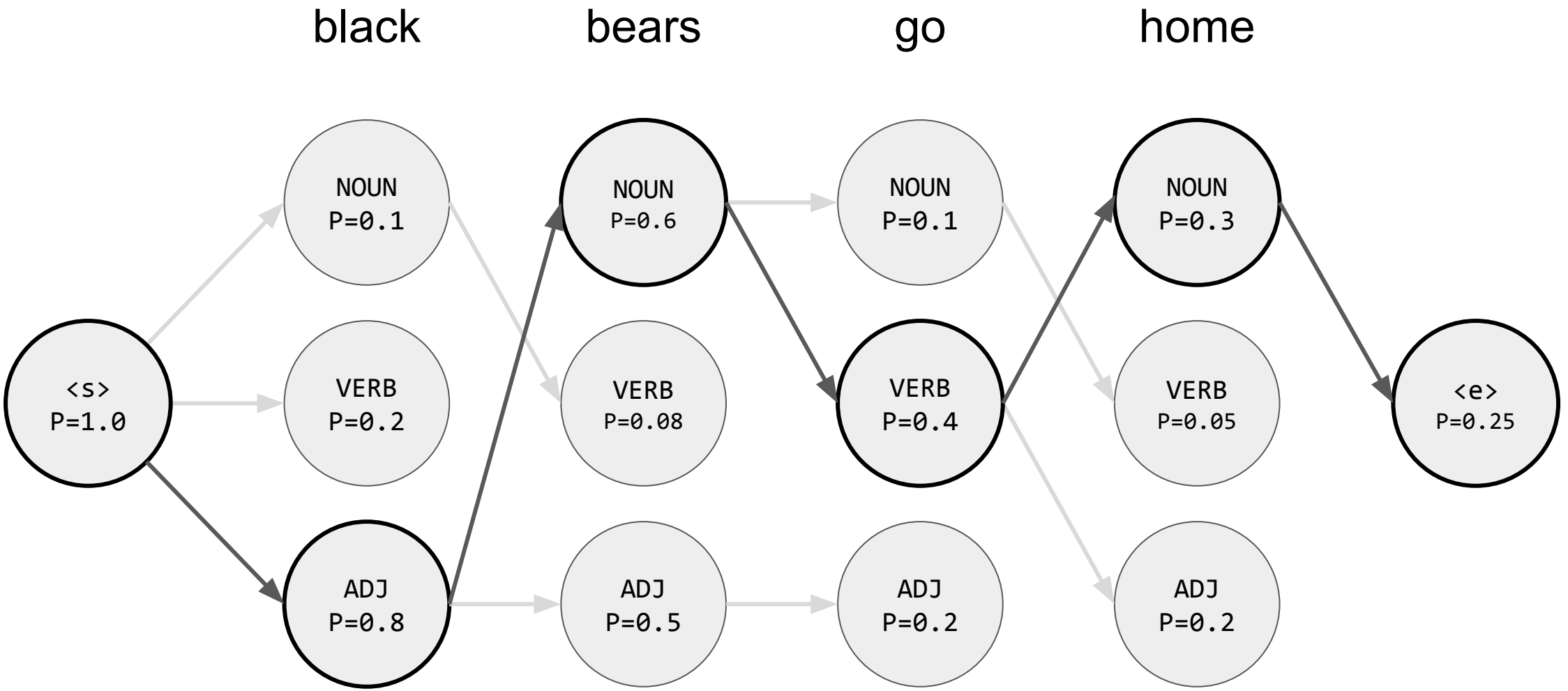
Viterbi Demo for “black bears go home”



Now we want to find the path that got us the maximum probability of P=0.25



Best path for “black bears go home”



Following the route backwards will give us the optimal path and the tags!

Parsing

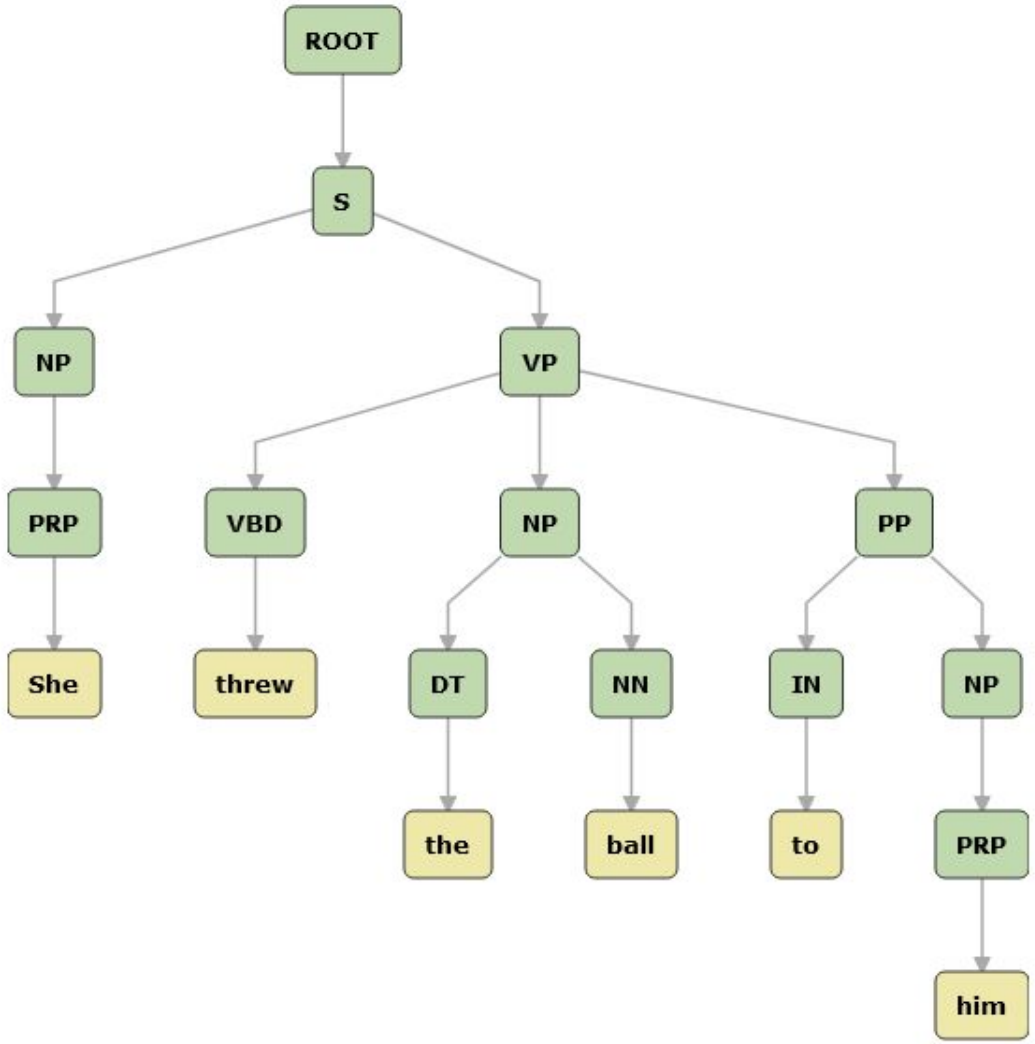
Constituency parsing

Breaking a sentence into noun phrases, verb phrases, etc

Noun Phrase (NP) - noun with determinants/adjectives
- The small dog
- five cars

Verb phrase (VP) - verb plus arguments (excluding its subject)
- passed David the salt
- walked away

Prepositional phrase (PP) - preposition plus object (noun phrase)
- To the trains
- On top of the Empire State Building



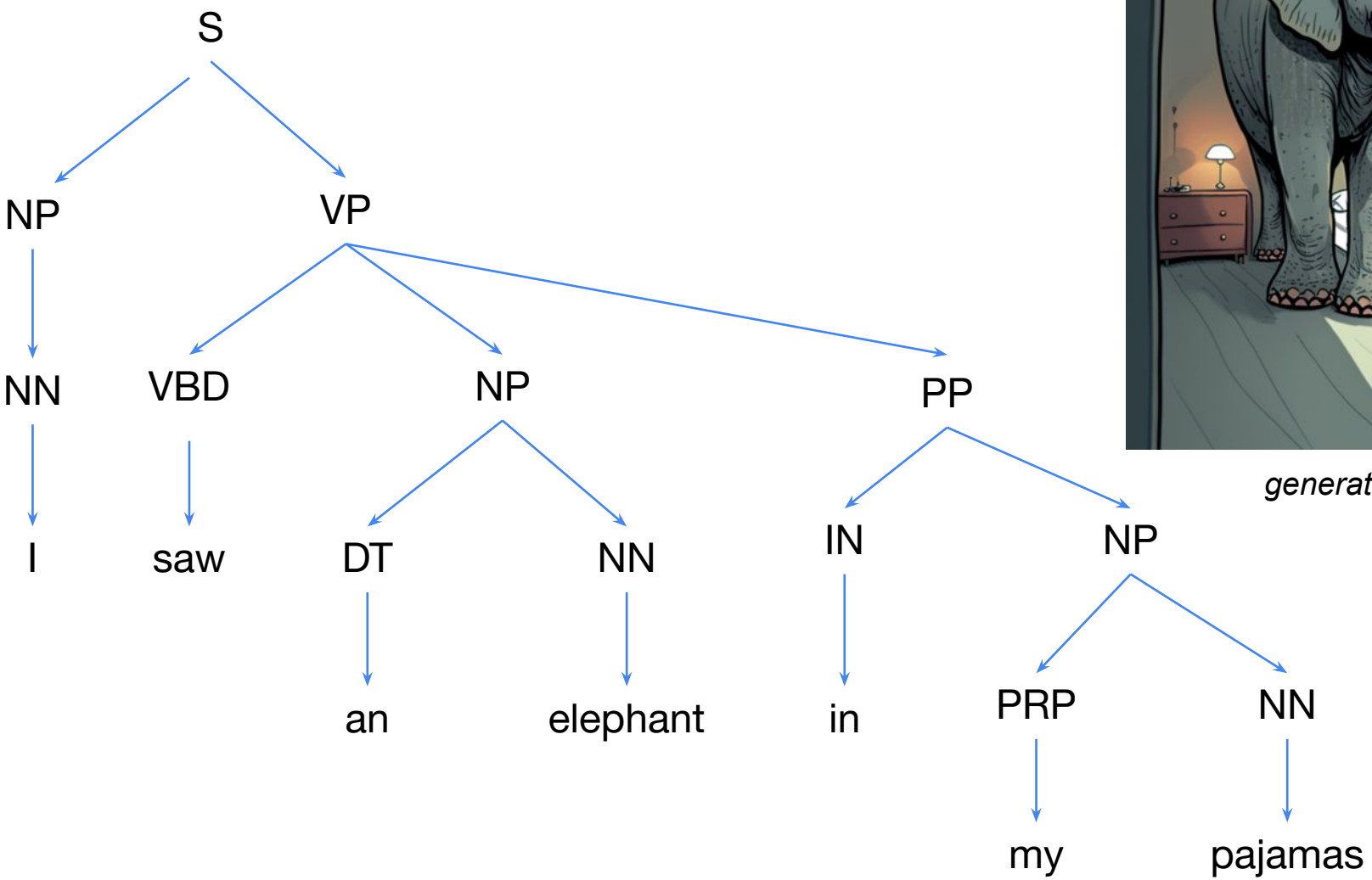
https://stanfordnlp.github.io/CoreNLP/demo.html

What is so hard about syntactic analysis? Ambiguity.

"I saw an elephant in my pajamas."

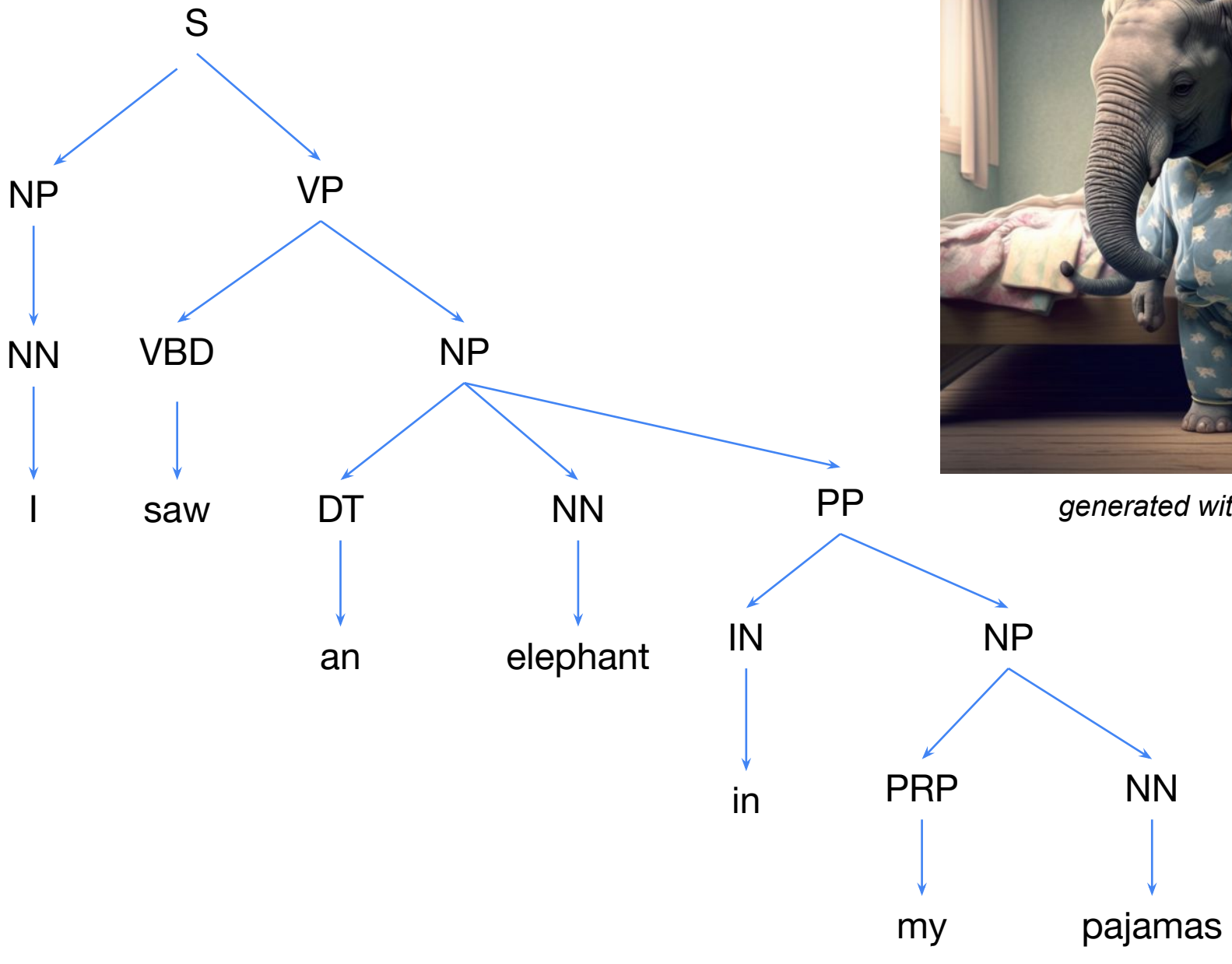
It is not uncommon for moderate length sentences - say 20 or 30 words in length - to have hundreds, thousands, or even tens of thousands of possible syntactic structures.

Ambiguity



generated with Midjourney

Ambiguity



generated with Midjourney

Why do we care about this syntactic structure?

- Many aspects of meaning can be learnt using the syntactic structure.
  - The NP preceding VP is likely the subject of the action.
  - The NP following the VP is likely the object of the action.

- Knowing basic units is helpful in modelling language.
  - You can use this to predict or complete the sentence (language modelling)
  - Re-organize sentences or simplify them (summarization)

- Many many NLP problems use sentence structure to make decisions
  - Relation extraction
  - Question answering
  - Machine translation
  - Semantic role labelling
  - …

Parsing: Overview

Types of Parsing:

- Constituency / phrase-structure
- **Dependency**
- Semantic / frame

Parsing Techniques / Algorithms:

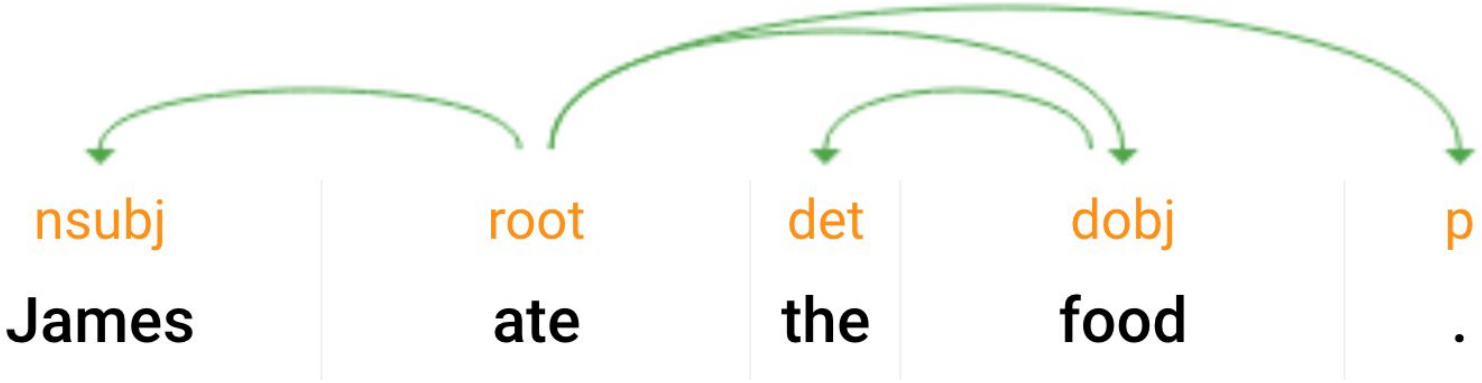
- **Transition-based**
- Graph-based
- Chart-based

Similar goal: Find a tree that represents how the tokens in a sentence are structured.

Different formalisms, but similar algorithms used for all types.

Dependency Parsing

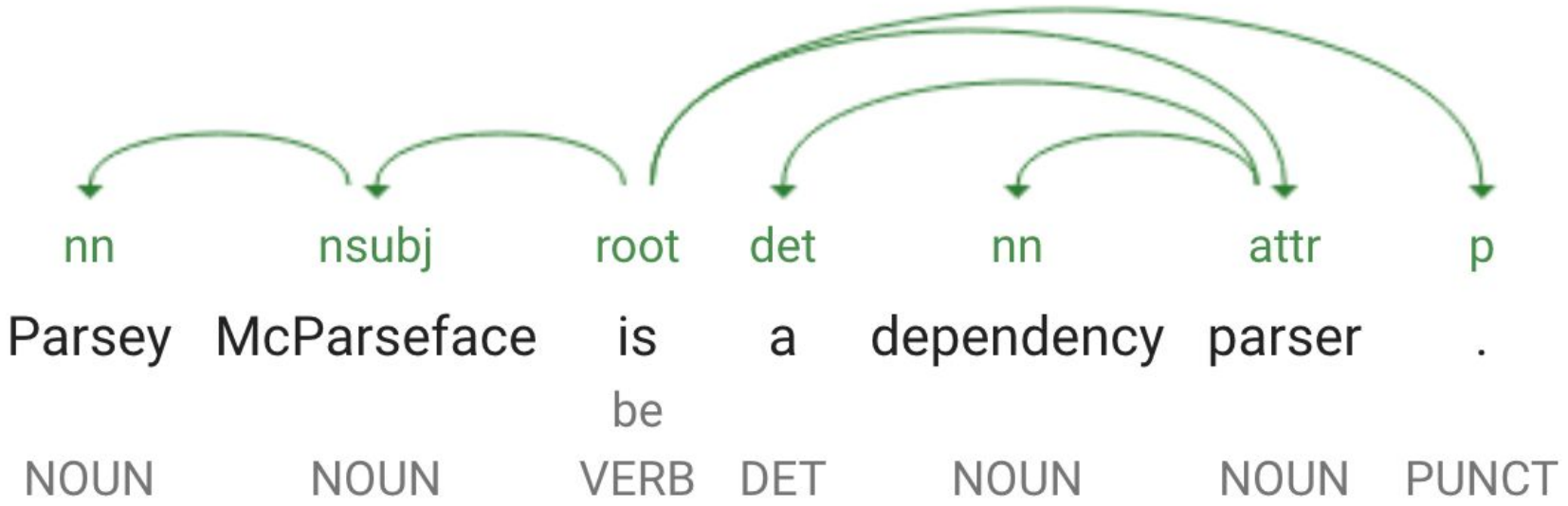
- Given a sentence, draw edges between pairs of words, and label them.
  - Result should be a tree
  - The edge-labels between word pairs should convey the correct syntactic relation.



Parsing!

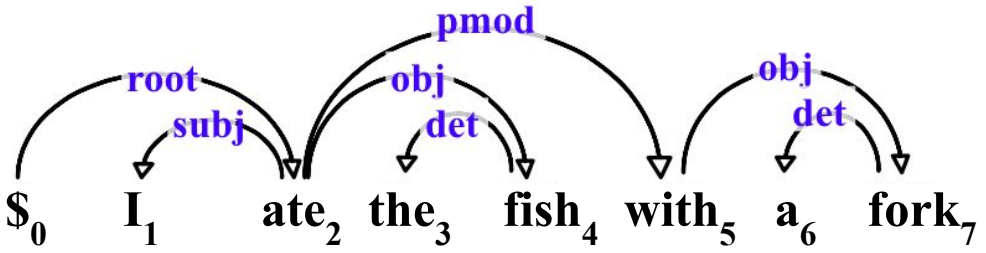
"How do I build the parse tree?"

- Not a single formalism / correct answer
- Often need training data to train a system

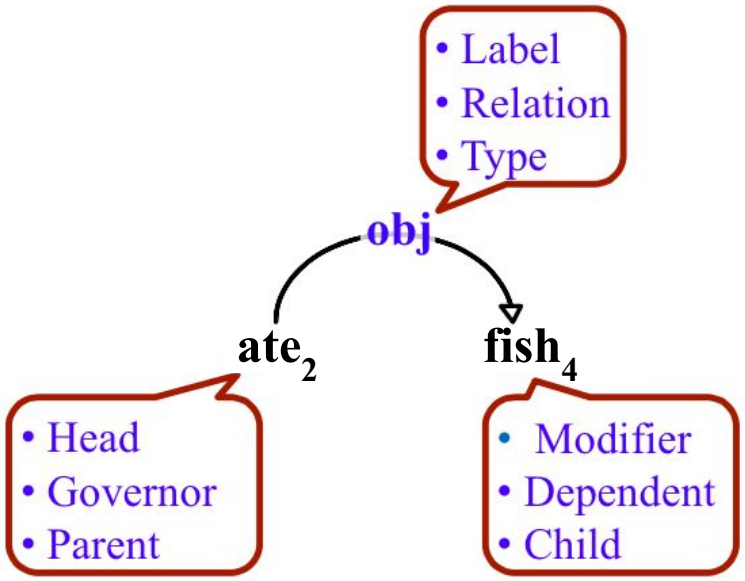


Formal definition of Dependency Parsing

- Formally, a dependency structure for a given sentence is a **directed graph** originating out of a unique and artificially inserted **root node**, which we always insert as the leftmost word.



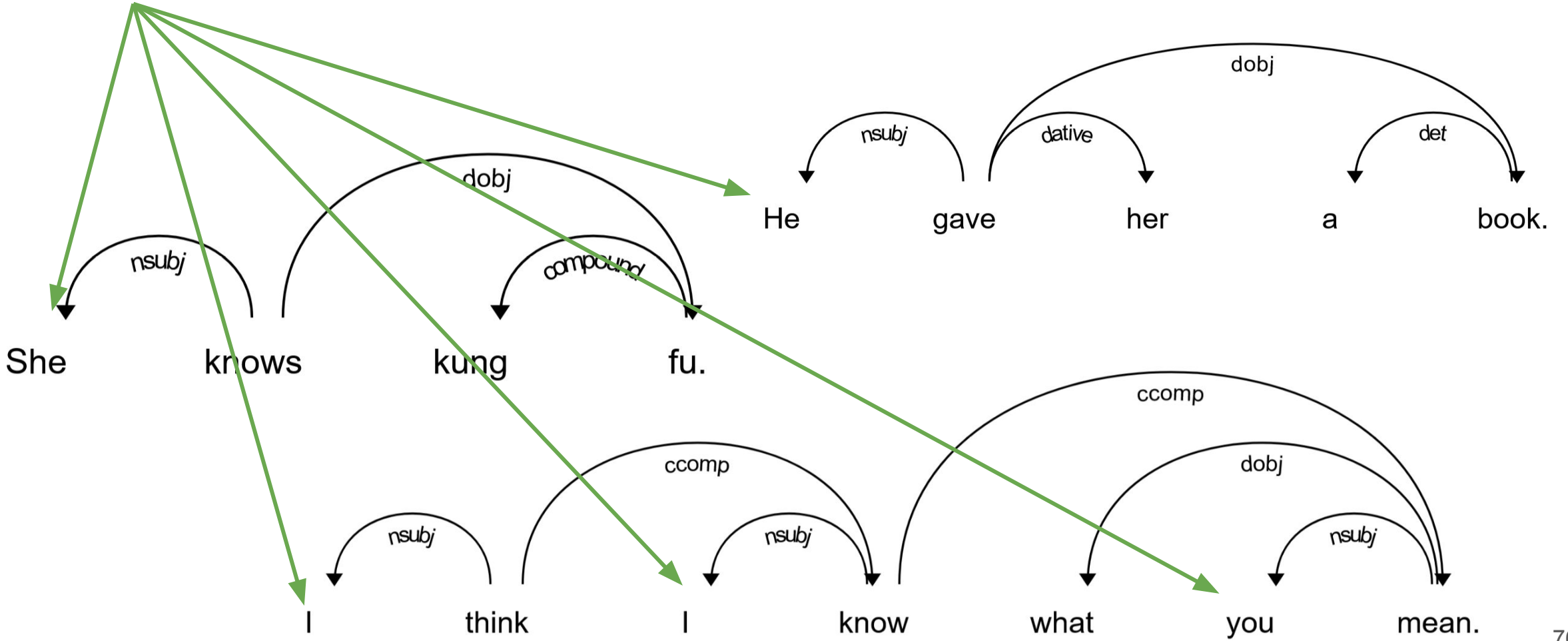
- In the most common case, every valid dependency graph has the following properties:
  - Each word has exactly **one incoming edge** in the graph (except the root, which has no incoming edge)
    - Weakly **connected** (replacing its edges with undirected edges results in a connected graph)
    - **Acyclic**



Some linguistic phenomena you can capture

Who did what to whom:

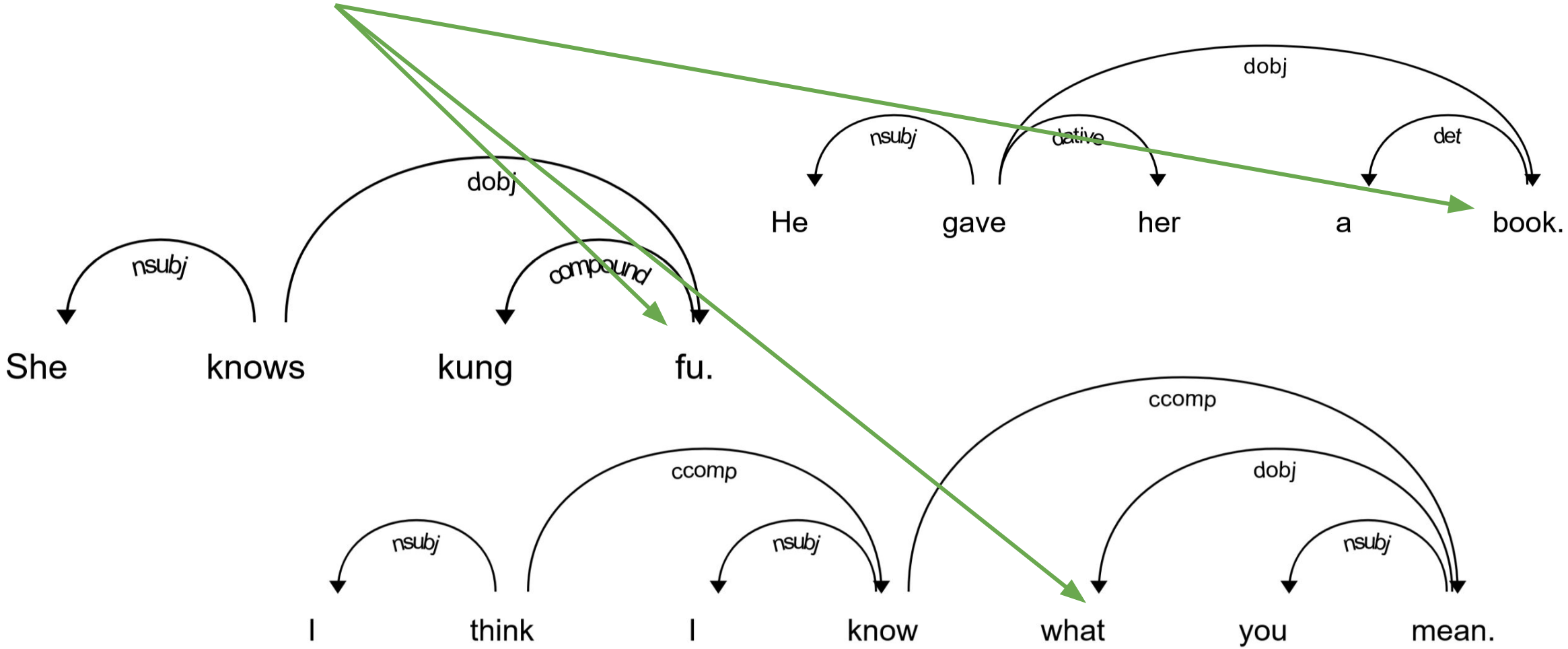
- Subject, direct object, indirect object



Some linguistic phenomena you can capture

Who did what to whom:

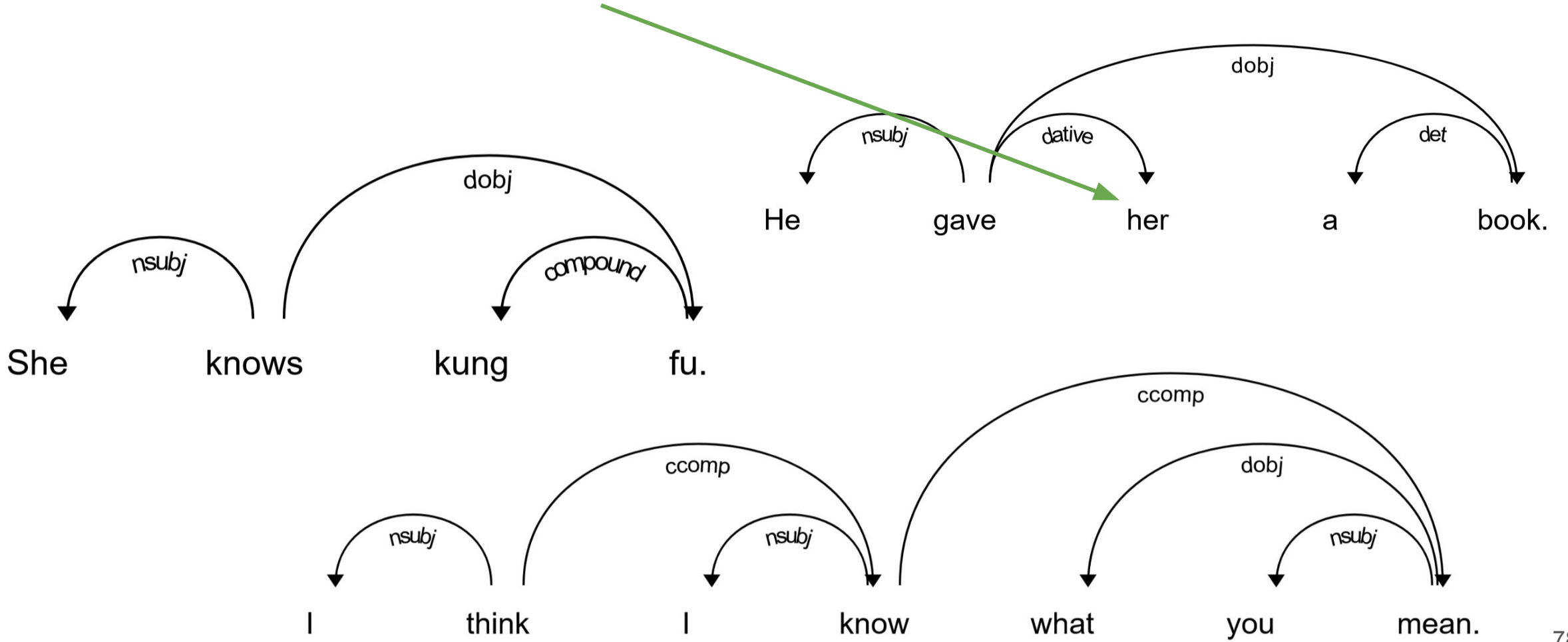
- Subject, direct object, indirect object

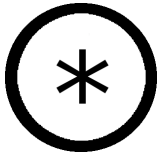


Some linguistic phenomena you can capture

Who did what to whom:

- Subject, direct object, indirect object





Breather

Transition based parsing

Mechanism: build a parse tree with a sequence of actions (transitions)
Set-up (**state**):

- A **buffer** initialized with the words
- A **stack** to store state:
  - Words under consideration for more edges
- Dependency **arcs** (edges)
  - The partially constructed tree

Transition based parsing

Mechanism: build parse tree by sequence of actions (transitions)

Arc-Standard parsing:

- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge

Arc-Standard Parsing Example

Arc-Standard parsing:

- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge

Arc-Standard Parsing Example

Arc-Standard parsing:

- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge

Arc-Standard Parsing Example

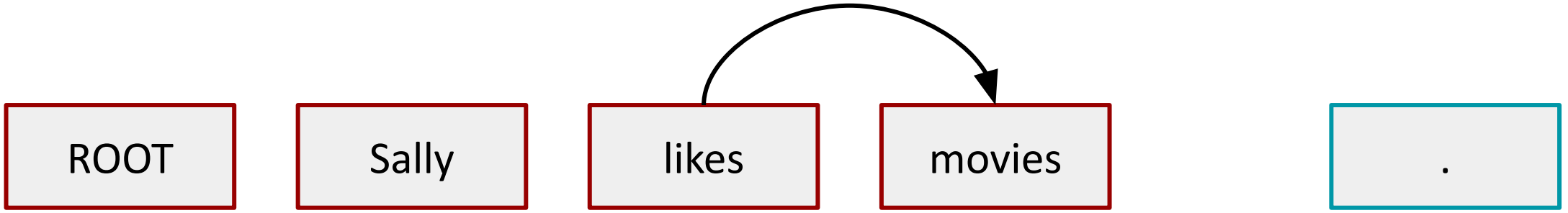
Arc-Standard parsing:

- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge

Arc-Standard Parsing Example

Arc-Standard parsing:

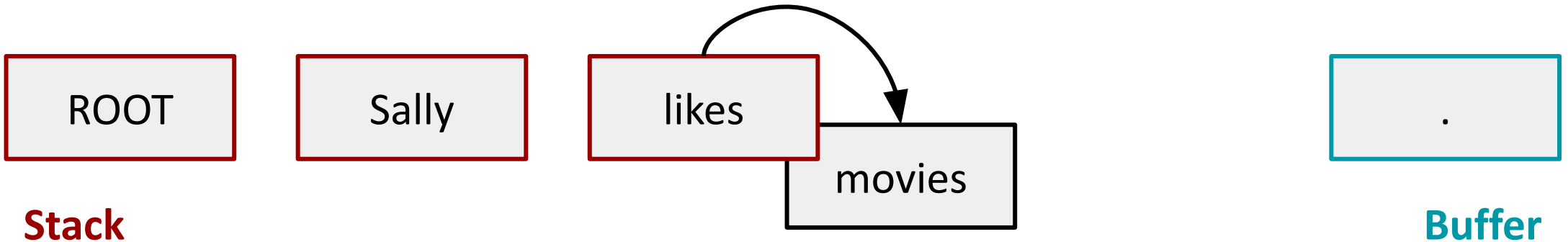
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge
  - Remove child from stack



Arc-Standard Parsing Example

Arc-Standard parsing:

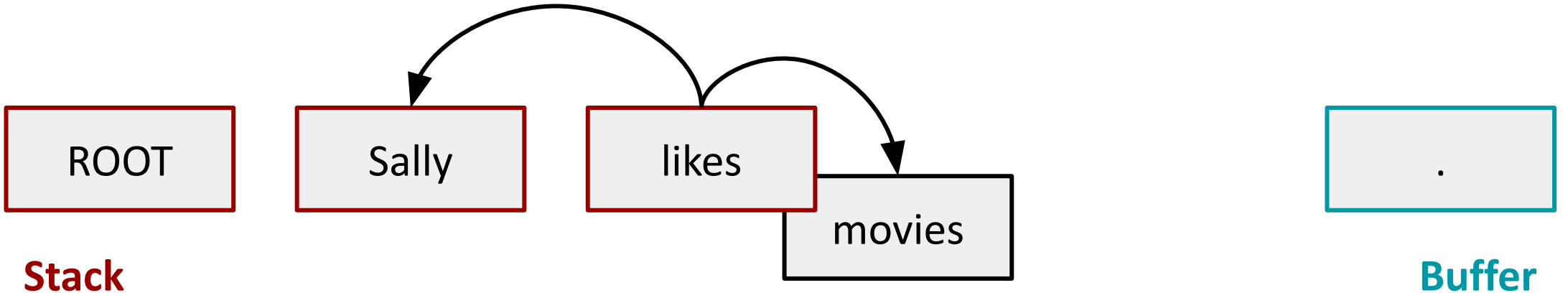
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge
  - Remove child from stack



Arc-Standard Parsing Example

Arc-Standard parsing:

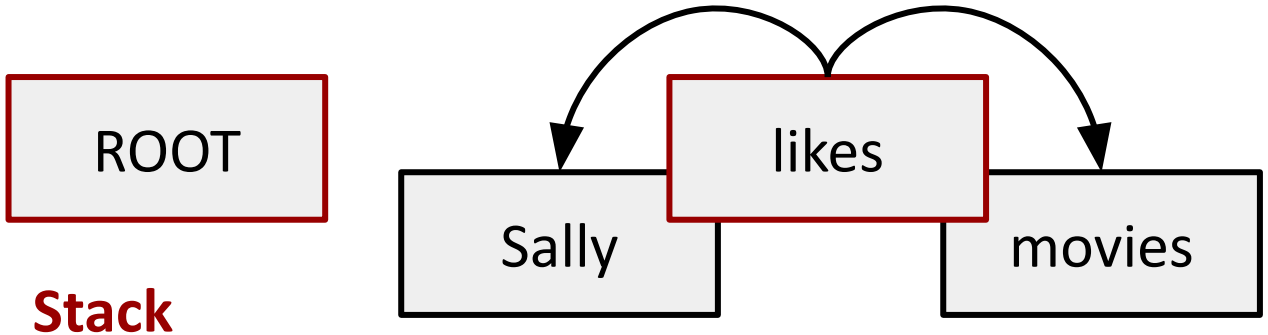
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
  - Remove child from stack
- **Right-arc:** add right edge



Arc-Standard Parsing Example

Arc-Standard parsing:

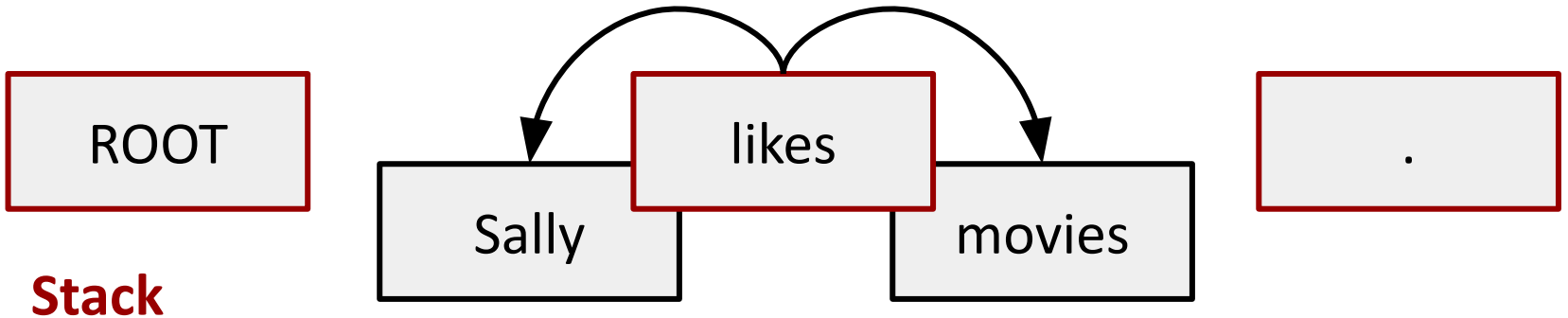
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
  - Remove child from stack
- **Right-arc:** add right edge



Arc-Standard Parsing Example

Arc-Standard parsing:

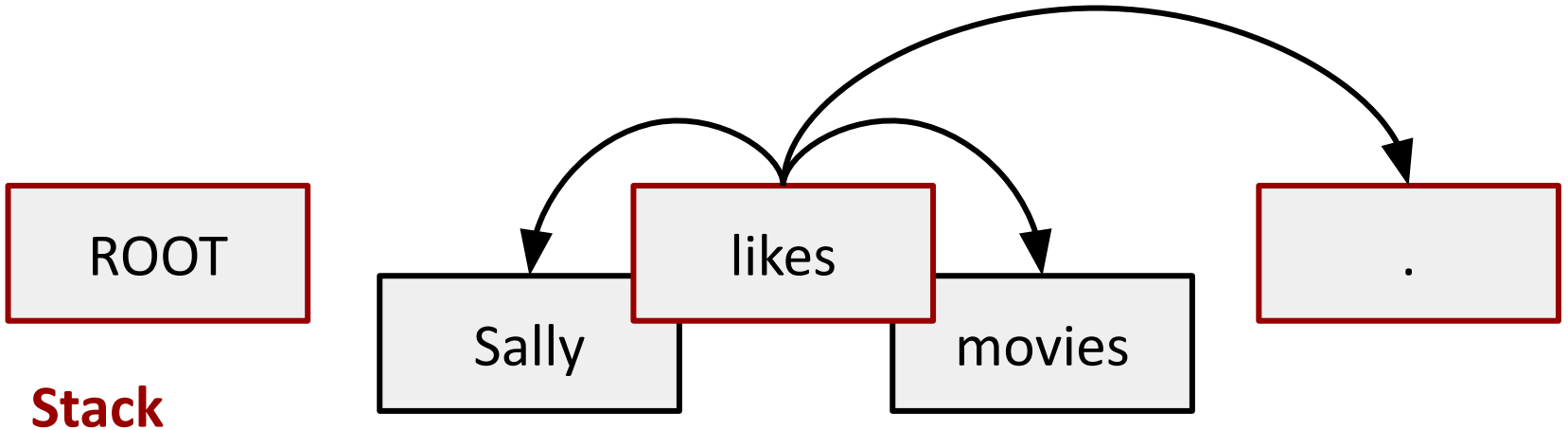
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge



Arc-Standard Parsing Example

Arc-Standard parsing:

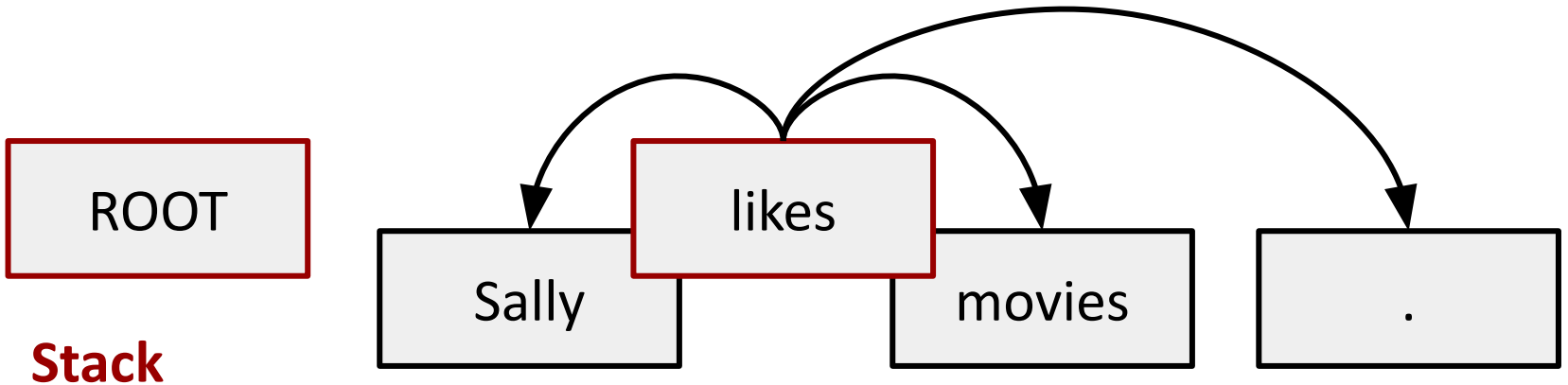
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge
  - Remove child from stack



Arc-Standard Parsing Example

Arc-Standard parsing:

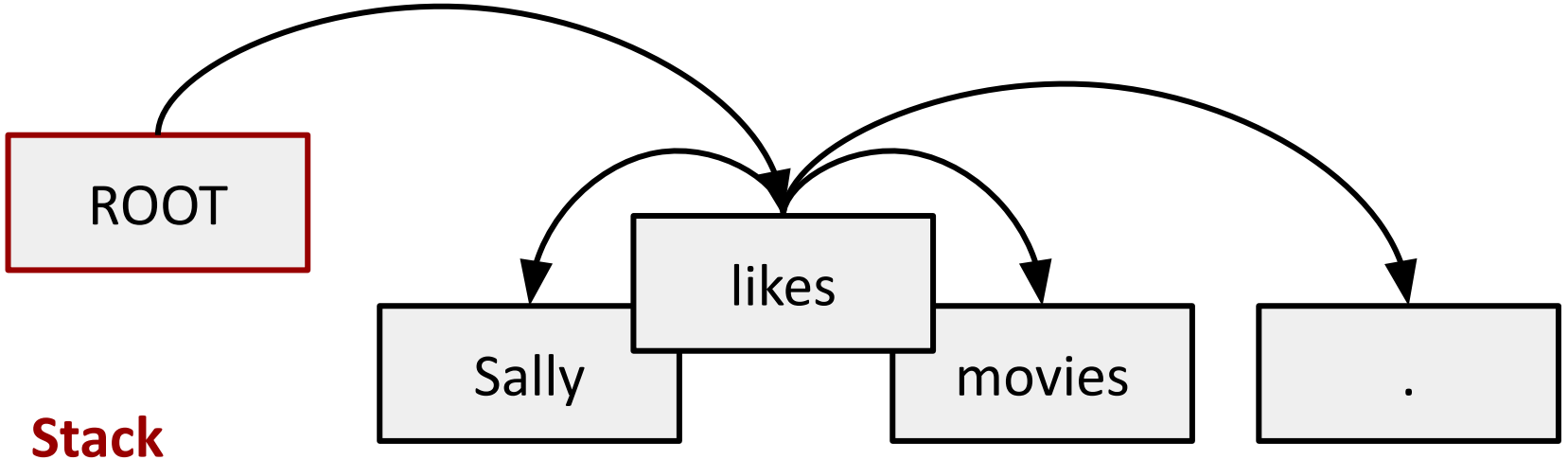
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge
  - Remove child from stack



Arc-Standard Parsing Example

Arc-Standard parsing:

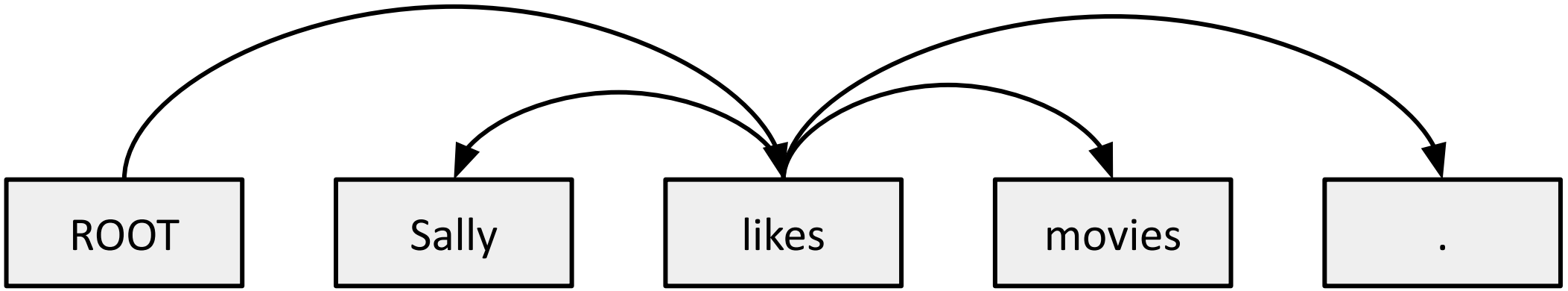
- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge
  - Remove child from stack



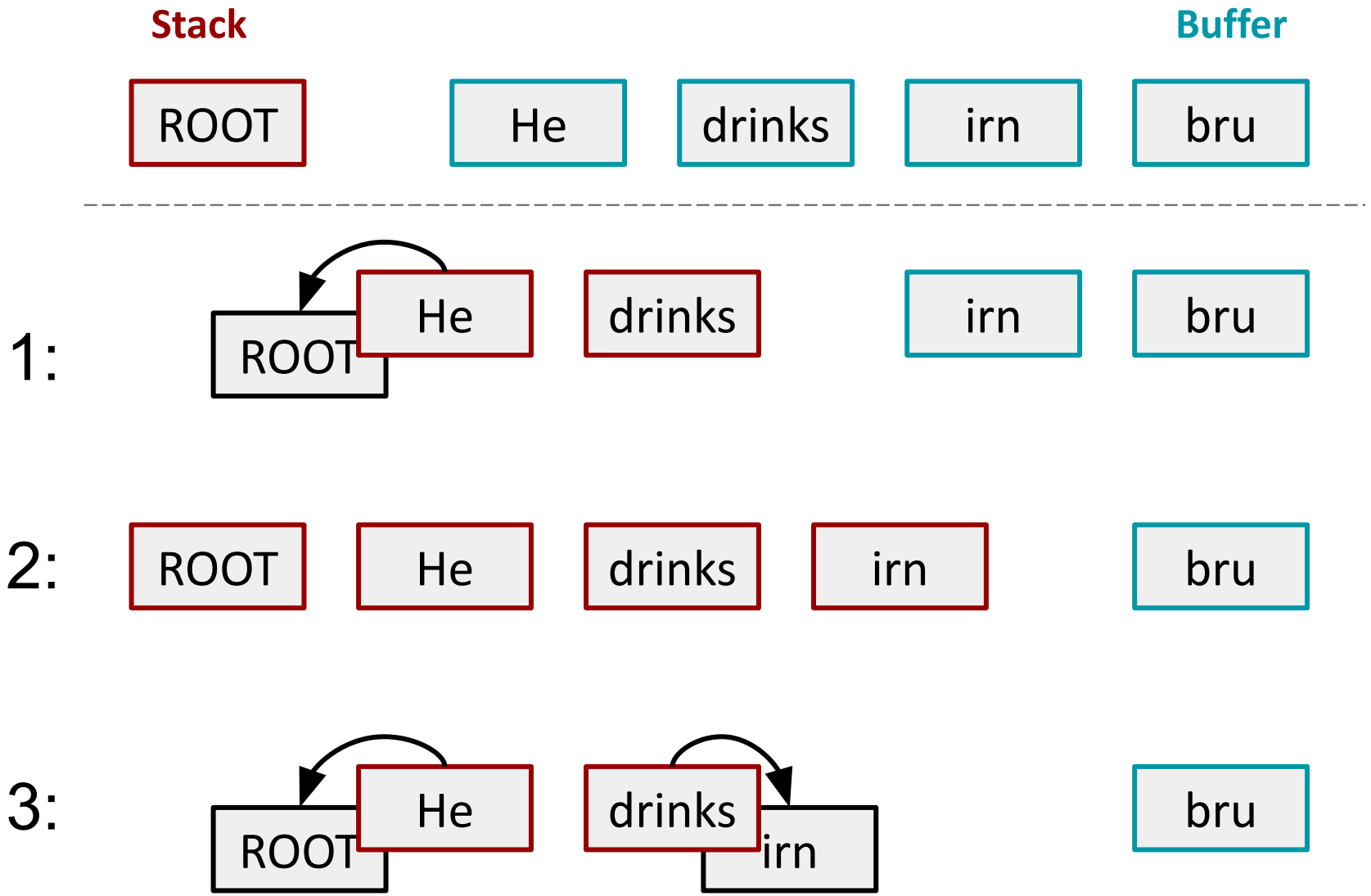
Arc-Standard Parsing Example

Arc-Standard parsing:

- **Shift:** pop from buffer, push to stack
- **Left-arc:** add left edge
- **Right-arc:** add right edge



Match the parsing result to the actions



A. Shift, Left-Arc, Shift

B. Shift, Shift

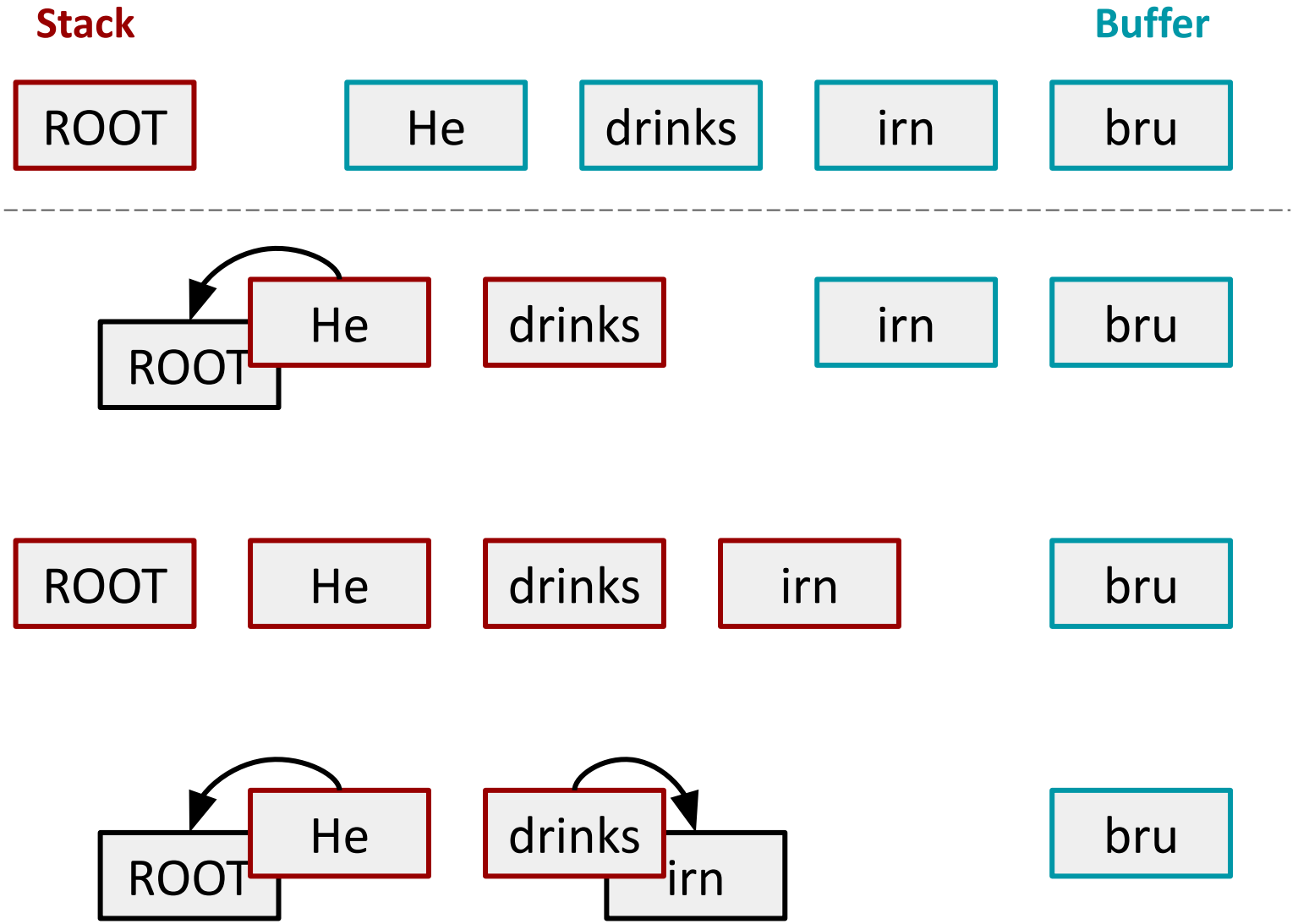
C. Left-Arc, Shift, Shift

D. Shift, Left-Arc, Shift, Shift, Right-Arc

E. Shift, Left-Arc, Right-Arc, Shift

F. Shift, Shift, Shift

Match the parsing result to the actions



A. **Shift, Left-Arc, Shift**

B. Shift, Shift

C. Left-Arc, Shift, Shift

D. **Shift, Left-Arc, Shift, Shift, Right-Arc**

E. Shift, Left-Arc, Right-Arc, Shift

F. **Shift, Shift, Shift**

Transition-Based Parsing: Analysis

"Shift-reduce" parsing:

- **Shift:** (*shift*) move from buffer to stack
- **Reduce:** (*left-arc, right-arc*) add edges, and remove elements from stack

For N tokens:

- O(N) shift operations (one for each token)
- O(N) reduce operations (one for each edge)

O(N) = linear time parsing! *(if you can choose the right transitions)*

Transition-Based Parsing: Analysis

"Shift-reduce" parsing:

- **Shift:** (*shift*) move from buffer to stack
- **Reduce:** (*left-arc, right-arc*) add edges, and remove elements from stack

Actions are *irreversible!* Cannot undo with other transitions.

- **Corollary:** must choose correct* action every time!

* *May be more than one correct action!* (spurious ambiguity)

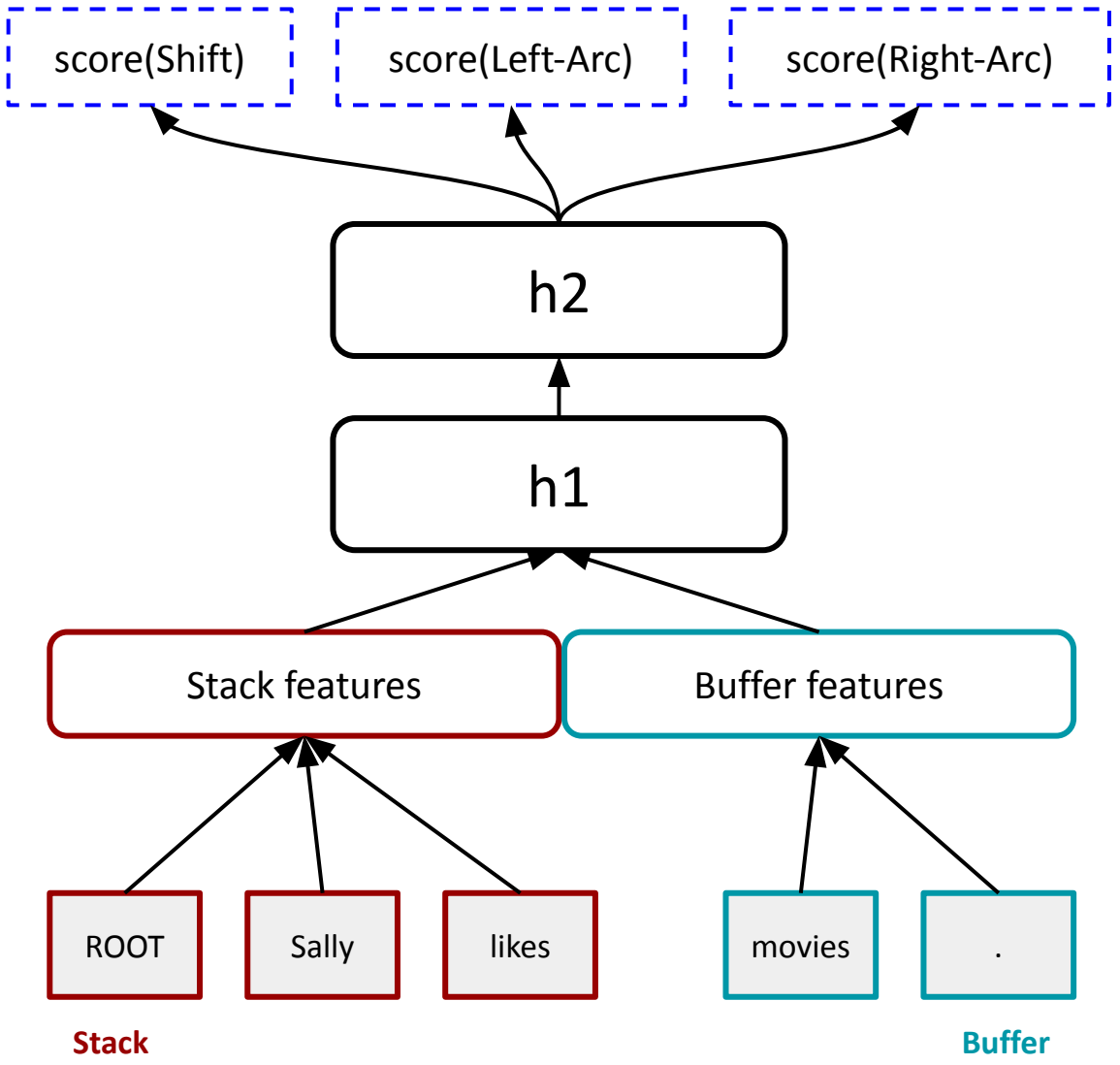
How do you choose the right one?

How to choose transitions?

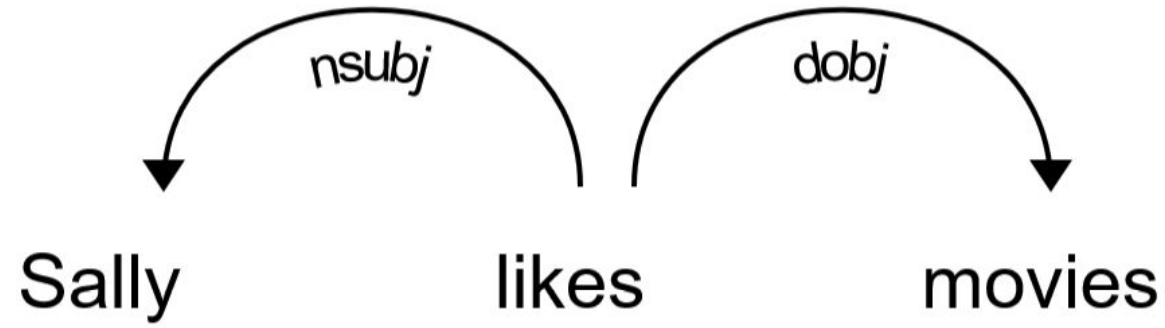
Multi-class prediction
Inputs: state = {stack, buffer, other features}
Targets: transitions = {shift, left-arc, right-arc}

Classifier: **score(a_i | $state_i$)** at step i
- Rule-based, SVM, NN, etc.

Training:
- Predict "gold" actions (*from true tree*)
- ***Or:*** "dynamic oracle" rewards good actions



What about the labels on the arcs?



- Use a separate classifier that picks arc label (given the stack features & buffer features)
  - Decide on subject, object, etc

State-of-the-art Results

Performance is very high, but progress is still being made!

Take away: New techniques for parsing now make it effective enough for real-world applications!

Rank	Model	LAS ↑	UAS	POS	Paper	Code	Result	Year
1	**Label Attention Layer + HPSG + XLNet**	96.26	97.42	97.3	Rethinking Self-Attention: Towards Interpretability in Neural Parsing			2019
2	**DMPar + XLNet**	95.92	97.30		Enhancing Structure-aware Encoder with Extremely Limited Data for Graph-based Dependency Parsing			2022
3	**ACE**	95.8	97.2		Automated Concatenation of Embeddings for Structured Prediction			2020
4	**Deep Biaffine + RoBERTa**	95.75	97.29		Deep Biaffine Attention for Neural Dependency Parsing			2016
5	**HPSG Parser** (Joint)	95.72	97.20	97.3	Head-Driven Phrase Structure Grammar Parsing on Penn Treebank			2019
6	**MFVI**	95.34	96.91		Second-Order Neural Dependency Parsing with Message Passing and End-to-End Training			2020
7	**CVT + Multi-Task**	95.02	96.61		Semi-Supervised Sequence Modeling with Cross-View Training			2018
8	**RNG Transformer**	95.01	96.66		Recursive Non-Autoregressive Graph-to-Graph Transformer for Dependency Parsing with Iterative Refinement			2020
9	**SpanRel**	94.70	96.44		Generalizing Natural Language Analysis through Span-relation Representations			2019
10	**CRFPar**	94.49	96.14		Efficient Second-Order TreeCRF for Neural Dependency Parsing			2020

https://paperswithcode.com/sota/dependency-parsing-on-penn-treebank

Parsing Summary

- Syntactic parsing aims to find phrase-structure in sentences and word-word relations.

- Dependency parsing
  - Word-word relations e.g., nsubj(ate, Elephant).
  - Parsing generates dependency trees by predicting edges or pruning edges.
  - Transition-based parsing formulates edge prediction as a classification task.
    - Predict one of four actions to take next.

- Parsing is a widely used step in constructing deeper, semantic representations.

Beyond Syntax

There have been efforts to build ***semantic*** structures that go beyond syntax.

Example: Abstract Meaning Representations

"The London emergency services said that 11 people had been sent to hospital."

```
(s / say-01
   :ARG0 (s2 / service
      :mod (e / emergency)
      :location (c / city :wiki "London" :name (n / name :op1 "London")))
   :ARG1 (s3 / send-01
      :ARG1 (p / person :quant 11)
      :ARG2 (h / hospital)))
```

Parsing these is much more difficult – it's a graph (not just a tree).

Beyond Syntax

There have been efforts to build ***semantic*** structures that go beyond syntax.

Example: Abstract Meaning Representations

"The London emergency services said that 11 people had been sent to hospital."

```
(s / say-01
    :ARG0 (s2 / service
              e / emergency)
              ion (c / city :wiki "London" :name (n / name :op1 "London")))
    :ARG1 (s3 / send-01
        :ARG1 (p / person :quant 11)
        :ARG2 (h / hospital)))
```

Verb disambiguation: PropBank

Named Entity Recognition

Parsing these is much more difficult – it's a graph (not just a tree).

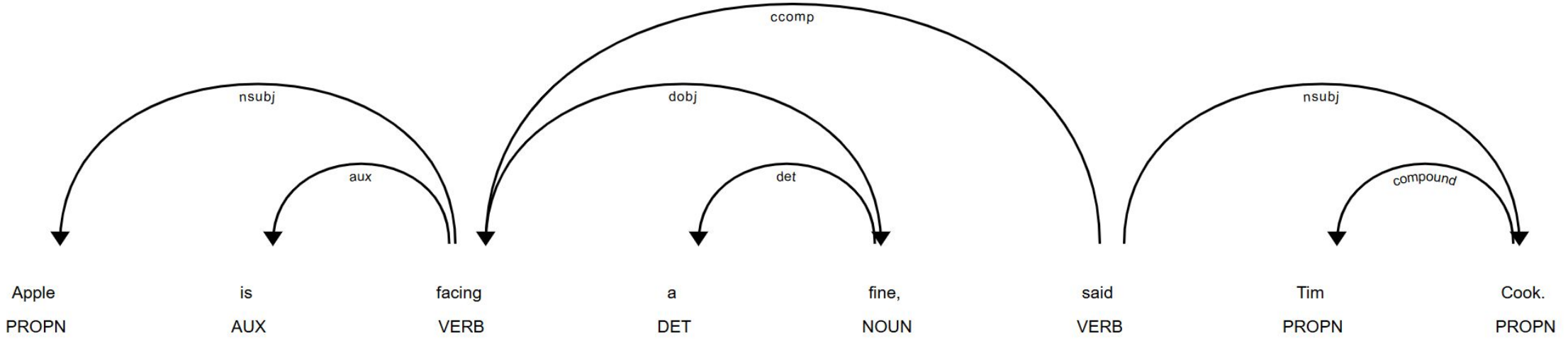
Parses and part of speech help distinguish meaning

Example: What if I worked at Bloomberg and wanted to extract company-risk relations from news archives?

Two sentences:

- *Apple is facing a fine, said Tim Cook.*
- *I feel fine, said Tim Cook of Apple.*

A simple bag-of-words classifier would mistake these as the same.



NLP Summary

- NLP provides us with tools to model language beyond simple lexical (word) features.

- Sequence models like HMMs allow us to label break apart patterns in language.
  - Part-of-speech tags
  - Sentence or word breaks

- Tree structured prediction
  - Predict complex patterns in language (We saw dependency parsing)
  - Helpful for advanced applications (QA, Dialogue Systems, etc.) that we'll see in coming lectures.

- Recent neural models can be effective even without these features, though.

Use the lab time to finish the labs

HUGGING FACE

- There is a Mini-Lab for this lecture next week
  - It is examinable
- Don’t forget to get going on your coursework!
- Come on Tuesdays and work on the labs & coursework